

The Problem

Fraction reducing or simplifying fractions means keeping the fraction in its most simplified form by dividing the numerator (top part) and the denominator (bottom part) by the largest number that can divide both exactly. You have to consider the following scenarios;

- If the fraction is a proper fraction (i.e. numerator is smaller than the denominator), the simplest form has to be produced. E.g., $3/18$ has to be simplified as $1/6$.
- If the fraction is improper, convert it into a mixed number and simplify the proper fraction part.

E.g.:- (i) $39/18$ is simplified as $2\frac{1}{6}$
(ii) $24/3$ is simplified as 8.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: Input file will contain a single line where the numerator and denominator is separated by a '/' sign (front slash).

Output: When writing the output, for a proper fraction, the same format above should be followed. When writing a mixed number, the whole number should be separated by the fraction part with a '|' sign (pipe sign)

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	39/18	2 1/6
input_2.txt	48569/65711	17/23

The Problem

FRACTRAN is an esoteric programming language. That is, a programming language that nobody uses.

A FRACTRAN program is a list of fractions. In this problem, you are going to use the following FRACTRAN program:

$$[[455, 33], [11, 13], [1, 11], [3, 7], [11, 2], [1, 3]]$$

You are given an integer n . Iterating through the list above from the start to the end, for each pair p in the list, you check whether $(n \cdot p[0])$ is a multiple of $p[1]$, and do the following:

- If yes, then replace n by $n \cdot p[0] / p[1]$, go back to the beginning and repeat.
- If the condition doesn't hold for any of the pairs, stop.

Develop a Python program to take n as the input, simulate the above FRACTRAN program and output the number of times n changed and the last value of n .

Format:

Input: An integer denoting the value of n

Output: Two integers separated by a space: the number of times n changed and the last value of n .

Sample:

Case 1

Input: 2

Output: 2 1

Explanation:

Here $n=2$, the first pair to satisfy the condition is $[11, 2]$. Hence n becomes 11. Then the first pair to satisfy the condition is $[1, 11]$. Hence n becomes 1. For $n=1$, none of the pairs satisfy the condition, stop there. The changes of n : $2 \rightarrow 11 \rightarrow 1$, so n changed 2 times and the last value was 1.

Case 2

Input: 6

Output: 6 5

Explanation:

n is changed in the following pattern: $6 \rightarrow 33 \rightarrow 455 \rightarrow 385 \rightarrow 35 \rightarrow 15 \rightarrow 5$

The Problem

Drivers are advised to keep two seconds of space between their car and the vehicle in front. This distance is the distance your car goes in two seconds at the current speed. Additionally, for long vehicles (length greater than 6 meters), the driver should leave one second space for each extra 3 meters.

Given as input the speed and length of the vehicle, and the distance to vehicle in front, develop a Python program to output if the driver is safe, and, print the required total distance.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The file contains 3 integers on separate lines in the following order: speed of your vehicle (in kilometers per hour), length of your vehicle (in meters), and distance to the vehicle in front of you (in meters).

Output: If the driver is safe the output should be "SAFE". If the distance is not sufficient output "DANGER" on first line and the required total distance in meters (rounded to nearest integer) on second line.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	50 13 100	SAFE
input_2.txt	85 6 40	DANGER 47

Explanation:

Case 1: As the length is 13 meters, 3 seconds should be kept for the extra 7 meters. So total distance should be of 5 seconds. Therefore the required distance is 70 meters and the driver is safe.

The Problem

Drivers are advised to keep two seconds of space between their car and the vehicle in front. This distance is the distance your car goes in two seconds at the current speed. Additionally, for long vehicles (length greater than 6 meters), the driver should leave one second space for each extra 3 meters.

Given as input the speed and length of the vehicle, and the distance to vehicle in front, develop a Python program to output if the driver is safe, and, print the required total distance.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string "INVALID" to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The file contains 3 integers on separate lines in the following order: speed of your vehicle (in kilometers per hour), length of your vehicle (in meters), and distance to the vehicle in front of you (in meters).

Output: If the driver is safe the output should be "SAFE". If the distance is not sufficient output "DANGER" on first line and the required total distance in meters (rounded to nearest integer) on second line.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	50 13 100	SAFE
input_2.txt	85 6 40	DANGER 47

Explanation:

Case 1: As the length is 13 meters, 3 seconds should be kept for the extra 7 meters. So total distance should be of 5 seconds. Therefore the required distance is 70 meters and the driver is safe.

The problem

A palindromic word (or palindrome) is a string that remains the same when its letters are reversed (that is, it remains the same when read from left-to-right or right-to-left)

Examples: madam, civic, reviver, rotor, refer, level, noon.

Develop a python program to take as input a word and output whether the input word is a palindrome or not. If not the program should identify the longest palindromic substring (whose length is at least 3) in the input word. For example even though "afternoon" is not a palindrome "noon" is a palindrome. If no such substring is found, output "There are no palindromes". Then the program should repeat this by taking the next input word and continuing in the same manner. The program will stop when the input word is "bye".

Note that the input must be a single word. You may handle unexpected inputs appropriately.

Example execution session:

Enter word : reference
longest palindrome : refer

Enter word : level
longest palindrome : level

Enter word : mouse
There are no palindromes

Enter word : Bye
Bye!

The Problem

Fraction reducing or simplifying fractions means keeping the fraction in its most simplified form by dividing the numerator (top part) and the denominator (bottom part) by the largest number that can divide both exactly. You have to consider the following scenarios;

- If the fraction is a proper fraction (i.e. numerator is smaller than the denominator), the simplest form has to be produced. E.g., $3/18$ has to be simplified as $1/6$.
- If the fraction is improper, convert it into a mixed number and simplify the proper fraction part.

E.g.:- (i) $39/18$ is simplified as $2\frac{1}{6}$

(ii) $24/3$ is simplified as 8.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: Input file will contain a single line where the numerator and denominator is separated by a '/' sign (front slash).

Output: When writing the output, for a proper fraction, the same format above should be followed. When writing a mixed number, the whole number should be separated by the fraction part with a '|' sign (pipe sign)

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	39/18	2 1/6
input_2.txt	48569/65711	17/23

The Problem

Janith is a kid who likes to play with his box of bricks. He puts the bricks on top of each other and builds stacks of different height. When he told his sister that he has built a wall out of bricks, his sister told that all stacks should be of same height in order for it to be a real wall. Develop a Python program to find the minimum number of bricks that Janith has to move to have a wall. You have to use all the bricks in the given stacks and discarding is not allowed.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named `getText` to read input from file. Handle invalid input properly and output the string `INVALID` to file and display a meaningful message on screen.
- Have a function named `showResult` to display the output on the screen and to write it to a file named `result.txt` in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The first line contains integer N , the number of test cases. The next $2N$ lines contain the N number of data sets where the first line of each data set contains the number of stacks that Janith has built. The second line indicates the heights of each stack.

Output: For each test data set output a line giving the minimum number of moves that Janith needs to perform. If it is impossible to make brick stacks of equal height, output `IMPOSSIBLE`.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	1 6 5 2 4 1 7 5	5
input_2.txt	2 4 5 2 4 1 7 5	IMPOSSIBLE 1

The problem

A hash function is any function that can be used to map data of arbitrary size (e.g., a string of t variable length) to data of fixed size (e.g., an integer). The values returned by a hash function are called hash values. Develop a program to take as input a hash value, h and a list of strings, and output which of the n input strings map to h when a string is hashed with the following steps:

(01) Both lowercase and uppercase letters are assigned values as follows (e.g. $a=1$, $b=2$...) any digit ~~is~~ assigned value i (e.g. '8' gets 8). Any other symbols in the string are ignored.

(02) Obtain the product p of the value in step 1

(03) The hash value is obtained as follows, where p is the product in step 2 above

$$\text{Hash value} = (p + 743) \% 8887$$

Example: "pp2" $16 \times 16 \times 2 = p = 215$

$$\text{hash value} = (215 + 743) \% 8887 = 1255$$

^{Q3} You must get file name as input.

Input: 1.txt
1255
3
Sri-Lanka
Programming
pp2

pp2.



Navigation

Home

- My home
- Site pages
- My profile

Current course

- In16-S1-CS1032 (104982)
 - Participants
 - Badges
 - General
 - 2 January - 8 January
 - 9 January - 15 January
 - 16 January - 22 January
 - 23 January - 29 January
 - 30 January - 5 February
 - 6 February - 12 February
 - Lecture 5 Slides
 - Example Code Visualizations from Lecture 5
 - Submission for Lab 5
 - Lab 5- Additional Exercise 1
 - Description
 - Submission
 - Edit
 - Submission view
 - Lab 5- Additional Exercise 2
 - 13 February - 19 February
- My courses

Administration

- Course administration
- My profile settings

Description Submission Edit Submission view

Lab 5- Additional Exercise 1

Requested files: Lab5_Ex1.py (Download)

Type of work: Individual work

Write a program to display the following pattern for a given number of lines.

```

. . * . .
. * * * .
* * * * *
```

Input: Positive integer indicating the number of lines

Output: Triangular pattern of asterisks(*) shown above.
(There are no spaces between asterisks in same line)

Example

Input:

5

Output:

```

. . . . * . . . .
. . . * * * . . .
. . * * * * * . .
. * * * * * * * .
* * * * * * * *
```



- ▶ 2 January - 8 January
- ▶ 9 January - 15 January
- ▼ 16 January - 22 January
 - Submission for Lab 3
 - ▶ Lab 3- Additional Exercise 1
 - ▶ Lab 3- Additional Exercise 2
 - Description
 - Submission
 - Edit
 - Submission view
- ▶ My courses

Administration

- ▶ Course administration
- ▶ My profile settings

media. One can't identify whether the owner of a particular profile is a female or male due to these weird names.

Saman has come up with a simple way to recognize some users' genders by their profile names. This is his method:

- If the number of distinct characters in one's profile name is odd and three letters "b", "o" and "y" can be found in the order, then the profile owner is a male. (Output: *He is a boy!*)
- If the number of distinct characters in one's profile name is even and four letters "g", "i", "r" and "l" can be found in the order, then the profile owner is a female. (Output: *She is a girl!*)
- If none of the above condition is true Saman can't identify gender of users by their profile names. (Output: *Cannot identify!*)

You are given the string that denotes the profile name; Help Saman to determine the gender of a user by his method. Provide him a program to accomplish this task.

PS: Please refer Section 5: Loop Control Structures in the CS1032 Course Notes.

Example**Case 1**

Input:

```
bigbroyaar
```

Output:

```
He is a boy!
```

Case 2

Input:

```
lonelygirl
```

Output:

```
She is a girl!
```

Case 3

Input:

```
funnyboy
```

Output:

```
Cannot identify!
```




tar [options]
filename

Store and extract files from a tarfile (.tar) or tarball (.tar.gz or .tgz).

top

Displays the resources being used on your system. Press q to exit.

touch filename

Create an empty file with the specified name.

who [options]

Display who is logged on.

Description

Submission

Edit

Submission view

Lab 3- Additional Exercise 2

Requested files: Lab3_Ex2.py ([Download](#))**Type of work:** Individual work

These days we can find weird profile names in all sort of social media. One can't identify whether the owner of a particular profile is a female or male due to these weird names.

Saman has come up with a simple way to recognize some users' genders by their profile names. This is his method:

- If the number of distinct characters in one's profile name is odd and three letters "b", "o" and "y" can be found in the order, then the profile owner is a male. (Output: *He is a boy!*)
- If the number of distinct characters in one's profile name is even and four letters "g", "i", "r" and "l" can be found in the order, then the profile owner is a female. (Output: *She is a girl!*)
- If none of the above condition is true Saman can't identify gender of users by their profile names. (Output: *Cannot identify!*)

You are given the string that denotes the profile name; Help Saman to determine the gender of a user by his method. Provide him a program to accomplish this task.

PS: Please refer Section 5: Loop Control Structures in the CS1032 Course Notes.

Example

Case 1

Input:

```
bigbroyaar
```

Output:

```
He is a boy!
```

Case 2

Input:

```
lonelygirl
```

Output:



Navigation

Home

- My home
- Site pages
- My profile
- Current course
 - In16-S1-CS1032 (104982)
 - Participants
 - Badges
 - General
 - 2 January - 8 January
 - 9 January - 15 January
 - 16 January - 22 January
 - 23 January - 29 January
 - 30 January - 5 February
 - 6 February - 12 February
 - 13 February - 19 February
 - Lecture 6 Slides
 - Sample Code Visualization for Functions
 - Coding Challenges (HackerRank)
 - Coding Challenges (TopCoder)
 - Submission for Lab 6
 - Lab 6- Additional Exercise 1
 - Lab 6- Additional Exercise 2**
 - Description
 - Submission
 - Edit
 - Submission view
- My courses

Administration

- Course administration
- My profile settings

Description Submission Edit Submission view

Lab 6- Additional Exercise 2

Requested files: Lab6_Ex2.py (Download)

Type of work: Individual work

Develop a Python program, which takes as input an English letter, and prints a text pattern on n lines as shown below, where

$n = \text{the height of the pattern} = (2 \times \text{position of the letter in the English alphabet} - 1)$.

The input is either a lower case or upper case English letter, while the output should be only in upper case. The numerical value of the ASCII code for 'A' is 65 and for 'a' is 97.

Example

Input:

E

Output

```

.....A.....
...ABA...
..ABCBA..
.ABCDCBA.
ABCDEDCBA
.ABCDCBA.
..ABCBA..
...ABA...
.....A.....

```

The Problem

Friday the 13th is considered an unlucky day in Western superstition. It occurs when the 13th day of the month in the calendar falls on a Friday. It occurs at least once every year, and up to three times a year.

Develop a Python program that takes as input a time period and outputs the number of times 13th would fall on a Friday in that period. The period is specified by start year, day of the week 1st day falls on that year, and the end year. The time period to be considered is from 1st January of start year to 31st December of end year.

Day of the week is specified as a number; the mapping between the number and the day will be as follows.

Sunday→0	Monday→1	Tuesday→2	Wednesday→3
Thursday→4	Friday→5	Saturday→6	

To determine whether a year is a leap or a not, you can use the following algorithm:

```
if (year is not divisible by 4) then (it is a common year)
else if (year is not divisible by 100) then (it is a leap year)
else if (year is not divisible by 400) then (it is a common year)
else (it is a leap year)
```

Format:

Input: Two input lines specifying the time period. The first line contains two integers separated by a space denoting start year and the day of 1st of January. The second line contains a single integer, the end year.

Output: A single integer (the number of times 13th would fall on a Friday in the period specified by the input).

Sample:

Case 1

Input: 2002 2
2004

Output: 5

Case 2

Input: 1863 4
1890

Output: 48

Case 3

Input: 1954 5
1960

Output: 12

date [options]	Display or set the system date and time.
----------------	--

df [options]	Display used and available disk space.
--------------	--

du [options]	Show how much space each file takes up.
--------------	---

file [options] filename	Determine what type of data is within a file.
----------------------------	---

find [pathname] [expression]	Search for files matching a provided pattern.
------------------------------------	---

grep [options] pattern [filename]	Search files or output for a particular pattern.
---	--

kill [options] pid	Stop a process. If the process refuses to stop, use kill -9 pid.
-----------------------	--

less [options] [filename]	View the contents of a file one page at a time.
------------------------------	---

ln [options] source	Create a shortcut.
------------------------	--------------------

Instructions:

1. Name your source file as "p1-<StudentNum>.py" and save it in a folder/directory named "p1" in your *home* directory.
 2. Upload the source file, "p1-<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "150001C" then the name of your source file will be "p1-150001C.py"

The Problem

Develop a Python program that takes as input a non-negative integer n , which consists of at least 2 and at most 100 digits. Then it should do the following:

- If n is divisible by 8, then output "Yes".
- If n is not divisible by 8, then try to obtain m , the largest number divisible by 8 by removing a minimum number of consecutive digits from n (but re-arranging digits is not allowed) and do one of the following:
 - If such a number m exists, then output "Yes" and the value of m .
 - If such a number m does not exist, then output "No".

You may handle unexpected inputs appropriately.

Example executions are shown below for $n=3454, 96, 10, 111111, 964$:

Enter n: 3454
Yes m=344

Enter n: 96
Yes

Enter n: 10
Yes m=0

Enter n: 111111
No

Enter n: 964
Yes m=96

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Index No.	
Name	
Group	
Date	

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "180001C" then the name of your source file will be "p1-180001C.py"
3. If you use IDLE, save your file in a folder named <Group name>_PP1_<Index number> in /nfs directory.
For example, E1_PP1_180001C. The file must be uploaded to VPL.
4. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

A matrix is called an $m \times n$ matrix if it has m rows and n columns. A $p \times q$ matrix and a $q \times r$ matrix can be multiplied to get a $p \times r$ matrix.

Define the i^{th} row and j^{th} column entry of matrix A to be a_{ij} .

Say A , B and C are matrices and $A_{p \times q} \times B_{q \times r} = C_{p \times r}$

Then $c_{ij} = \sum_{k=1}^q a_{ik} \times b_{kj}$

Example:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} -2 & 8 \\ -2 & 17 \end{pmatrix}$$

Develop a Python program to input two matrices and output the product of those two matrices.

Hint: Use `s.split()` to split a string `s` by spaces.

Format:

Input:

- The first line contains the number of rows (r_1) and the number columns (c_1) of the first matrix A
- The next r_1 lines: each contains c_1 number of integers separated by spaces
- The next line contains the number of rows (r_2) and the number columns (c_2) of the next matrix B
- The next r_2 lines: each contains c_2 number of integers separated by spaces

Output:

- If $c_1 \neq r_2$, output 'Invalid Input'
- If $c_1 = r_2$, output the product $A \times B$ matrix (has r_1 rows, c_2 columns) in the format same as the input

Sample:Case 1

Input: 2 3
1 2 3
4 5 6
3 2
1 0
0 1
-1 2
Output: -2 8
-2 17

Case 2

Input: 2 2
1 0
0 1
2 2
1 2
3 4
Output: 1 2
3 4

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

The Problem

The aim of any encryption is to distort the letters in the original message, so that only the intended party can make sense of it. In Caesar's cipher, a key (or an alternate alphabet) is used to replace the plain English letters. As an example consider the following scenario:

letter	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
key	T	H	E	Q	U	I	C	K	B	R	O	W	N	F	X	J	M	P	S	V	L	A	Z	Y	D	G

If you are asked to encrypt "Hello World!", using the above key, the resulting cipher text (distorted message) would be "Kuwwx Zxpwq!". This was obtained by replacing the plain letters, with the letters occurring in corresponding positions in the alternate alphabet (key), preserving the upper- or lower-case status. For e.g., 'h' is the 8th letter in plain alphabet and K is the letter occurring at 8th position in the key. Therefore, K is used to replace H. Similarly, 'e' is replaced by u.

Develop a Python program to read a cipher text as input, decode it using the above Key and output the original plain text message. You have to keep spacing and punctuations exactly as it was given. Only the letters have to change.

Format:

Input: A single line of cipher text.

Output: A single line of decoded text, formatted exactly as the input.

Sample:

Case 1

Input: Cxxq Wleol

Output: Good Luck!

Case 2

Input: Vkbs bs fxv ktpq ts bv suuns.

Output: This is not hard as it seems.

Case 3

Input: Fxz, vkbfcs tpu svtpvbfc vx ntou sufsu.

Output: Now, things are starting to make sense.

CS1032 Programming Fundamentals

2016 Batch, Jan - May 2017

Programming Assignment 1

Time: 1 hour only

Q10 [E10, Tues 1.15]

Index No.	
Name	
Group	
Date	

Password: E10BuJ@M

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "160001C" then the name of your source file will be "p1-160001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

Develop a Python program to read a list of positive integers as input from the keyboard and output whether the list contains duplicate numbers. Input numbers are separated by space and ends with a "-1". (This "-1" indicates the end of list and it is not one of the input numbers).

Your program must display the position of duplicate numbers in the list in sorted order. If there are n numbers in the list, then their positions are 1, 2, ..., n . You can assume that the maximum length of input list is 50. Print "None" if there are no duplicate values.

Format:

Input: A list of positive integers separated by a space, given in a single line. The list ends with -1.

Output: The positions of duplicate numbers in the list in sorted order (separated by a single space) or the string "None" exactly.

Sample:

Case 1

Input: 1 3 5 7 8 90 7 56 33 90 -1

Output: 4 6 7 10

Case 2

Input: 1 3 5 7 90 56 33 91 -1

Output: None

Case 3

Input: 5 5 5 5 1 5 5 -1

Output: 1 2 3 4 6 7

The Problem

The pattern for a vehicle license plate in Atlanta is XXNNXXN, where X stands for letters (upper case) and N for digits. Letters are in the range A - Z, while digits are 0 - 9. You are given a license plate string and an integer n. Develop a Python program to output the nth license plate string (in the same pattern), starting from the given license plate string.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: First line contains the given license plate string. The second line contains the integer n.

Output: A string representing the nth license plate, starting from the given license plate string.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	CA12E3 1	CA12E4
input_2.txt	AA00Y9 3	AA00Z2

CS1032 Programming Fundamentals

2016 Batch, Jan - May 2017

Programming Assignment 1

Time: 1 hour only

Q9 [E10, Tues 1.15]

Index No.	
Name	
Group	
Date	

Password: E10AE#q9

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "160001C" then the name of your source file will be "p1-160001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

A *palindromic number* (or *palindrome*) is a number that remains the same when its digits are reversed (that is, it remains the same when read from left-to-right or right-to-left).

Examples: 1, 2, ..., 9, 11, 22, ..., 99, 111, ..., 484, 676, 10201, 12321.

You are given a number and you have to determine if it is a palindrome or not. If not, you should add the value of the number read from left-to-right and its value read from right-to-left, and check if the obtained sum is a palindrome. If not, you should continue the process, until you eventually obtain a palindrome.

Develop a Python program that takes an integer n ($1 \leq n < 1000$) as input and outputs the palindrome you can eventually obtain as described above. If the number of addition operations exceeds 25 you should output "Too long".

Format:

Input: A positive integer

Output: Palindrome obtained through the above process or the string "Too long" exactly.

Sample:

Case 1

Input: 37

Output: 121

Explanation:

Since 37 is not a palindrome you will perform the addition: $37 + 73 = 110$. The number 110 is not a palindrome, either; so you will proceed to perform another sum: $110 + 011 = 110 + 11 = 121$. The obtained result is a palindrome, so you stop.

Case 2

Input: 7

Output: 7

Case 3

Input: 79

Output: 44044

Index No.	
Name	
Group	
Date	

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or other text editor.
Example: If your Index No. is "180001C" then the name of your source file will be "p1-180001C.py"
3. If you use IDLE, save your file in a folder named <Group name>_PP1_<Index number> in /nfs directory.
For example, E1_PP1_180001C. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

A **palindromic number** or numeral palindrome is a number that remains the same when its digits are reversed. Examples: 3, 22, 252, 4884, 16461, 6570756. The palindromic primes are prime numbers that are palindromic, such as 2, 3, 5, 7, 11, 101,..... (A prime number is a positive integer that can be divided only by 1 and the number itself; examples are: 2, 3, 5, 7, 11, 13, 17, 19.....). The palindromic squares are 0, 1, 4, 9, 121, 484, 676,..... (that is, squares of integers that are palindromic $0^2=0$, $1^2=1$, $2^2=4$, $3^2=9$, $11^2=121$, $22^2=484$, $26^2=676$).

Develop a Python program to output the sum of all palindromic squares and palindromic primes in a given range. Program input would be two positive integers **a** and **b** ($a \leq b$) separated by space. You should output the sum of all palindromic squares and palindromic primes inside the range **[a, b]**, that is in the range between **a** and **b**, inclusive.

Format:

Input: **a b**

(two positive integers **a** and **b** where $a \leq b$ and separated by space)

Output: the sum of all palindromic squares and palindromic primes in the range **[a, b]**

Sample:**Case 1**

input: 1 100
output: 42

Case 2

input: 400 1000
output: 6076

The Problem

A *hash function* is any function that can be used to map data of arbitrary size (e.g., a string of variable length) to data of fixed size (e.g., an integer). The values returned by a hash function are called *hash values*. Develop a Python program to take as input a hash value (an integer), h , and a list of strings, and output which of the input strings map to h when a string is hashed with the following steps.

1. Both lowercase and uppercase letters are assigned values as follows: $a=1, b=2, \dots, z=26$. Any digit "1" is assigned value 1 (e.g., "8" gets 8). Any others symbols in the string are ignored.
2. Obtain the product p of the values in step 1.
3. The hash value is obtained as follows, where p is the product in step 2 above.

$$\text{Hash value} = (p + 743) \% 8887$$

Example: If the string is "PP2" the values from step 1 are 16, 16, 2. Then p from step 2 is $16 \times 16 \times 2 = 512$. Therefore the hash value of "PP2", obtained in step 3, is: $(512 + 743) \% 8887 = 1255$.

- a) The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- b) Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- c) Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- d) The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The first line contains the hash value, h . The second line contains the number of strings, n . The next n lines contain a single string in each. ($h \geq 0, n > 0$)

Output: Print the strings that map to the given hash value, in the order they appear in the file. Each string should be on a new line. If no string maps to the given hash value output *NONE*

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	1255 3 Sri_Lanka Programming PP2	PP2

Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered

CS1032 Programming Fundamentals

2018 Intake, Jan - April 2019

Programming Problem 1

Time: 1 hour only

P10 [Fri 8.15]

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "180001C" then the name of your source file will be "p1-180001C.py"
3. If you use IDLE, save your file in a folder named <Group name>_PP1_<Index number> in /nfs directory.
For example, E1_PP1_180001C. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

Blackjack is a comparing card game between a player and a dealer. It is played with one deck of 52 cards. The objective of the game for the player is to win against the dealer in one of the following ways:

- Get 21 points on the player's first two cards (called a "Blackjack"), without a dealer blackjack
- Reach a final score higher than the dealer without exceeding 21

First the dealer deals 3 cards for himself and another 2 cards for the player. Based on them, you need to output whether the player "won", "lost" or should go for a "hit" (that means get another card from the deck).

The four suits are represented by their first letters: Diamonds→D, Hearts→H, Spades→S, Clubs→C.

Values represent by each card in the deck are as follows:

- Aces (A) can be either 1 or 11 points. (Decided by the player)
- Both 10 and face cards counted as 10 points. (10, J, Q, K)
- The rest of the cards represent the face value (2, 3, 4, 5, 6, 7, 8, 9)

Develop a Python program to output whether the player won, lose or should go for a hit based on the input cards of the player and dealer.

Format:

Input: Line 1: 3 cards of the dealer
Line 2: 2 cards of the player

Output: Whether the player "Won", "Lost" or should go for a "Hit"

Sample:

Case 1

Input: HA C5 D5
S7 C8
Output: Lost

Case 2

Input: H3 D2 C6
C8 HA
Output: Won



[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

2. Name your source file as "P2_Q5_E6.py" when you create/upload file in VPL. Comment your Index number on top of the Python source file. E.g :- # Index No: 160000A
3. If you use IDLE, save files in a folder named "p2" in your home directory. The file must be uploaded to VPL.
4. The sample input and files are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

Nimal has the largest stamp collection in his village and he likes to boast about his stamp collection by showing off at social events. Since Nimal is so sure about his superiority, he announces the number of stamps he has in his hand before an event. Kamala is a smarter kid who borrows stamps from her friends secretly and shows even a larger collection. She tries to borrow from all friends who are ready to lend. Develop a Python program to output the number of friends Kalama has to borrow from.

- a) The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- b) Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- c) Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- d) The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The first line contains integer N, the number of test cases. The next 2N lines contain the N number of data sets where first line of each data set contains the number of stamps that Kamala needs to borrow and number of friends ready to lend. The second line contains the number of stamps each friend is ready to offer.

Output: For each test data set, output a line giving the number of friends that Kamala needs to borrow stamps from. If there is no way to do it, output "IMPOSSIBLE".

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	1 100 6 13 17 42 9 23 57	6
input_2.txt	2 3 6 13 17 42 9 23 57 1000 3 314 159 265	3 IMPOSSIBLE

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

The Problem

Mr. Silva came across in a chemistry text book a chemical equation that was not balanced. Now, Mr. Silva suspects that some of the other equations might be erroneous as well. You have to help Mr. Silva in checking whether chemical equations are correct (balanced).

Develop a Python program that takes as input the left-hand side (LHS) and the right-hand side (RHS) of a chemical equation and outputs whether that equation is correct. Assume that we are looking at equations only with elements H, C, O and S and all the numbers would be single digit.

Note: You may use the Python built-in function "`split()`" in your program to process the input.

Hints: To split the LHS or RHS, `var` into compounds, use `var.split("+")`

To split for example `t=2*H2O`, use `t.split("*")`

Format:

Input: LHS and RHS of equation in two lines. The first line is for the LHS and the second for RHS. Each line will only have letters H, C, O or S, the symbols + and *, and digits.

Output: A single word denoting "Correct" or "Incorrect" as the answer. (Please print the exact text)

Sample:

Case 1

Input: `1*H2+2*O2`
`1*H2O`

Output: Incorrect

Case 2

Input: `2*H2+2*O2`
`2*H2O2`

Output: Correct

Explanation:

For Case 1, LHS has 2 H atoms and 4 O atoms while RHS has 2 H atoms and 1 O atom. Therefore the equation is not balanced.

For Case 2, LHS has 4 H atoms and 4 O atoms while RHS has 4 H atoms and 4 O atoms. Therefore the equation is balanced.

Index No.	
Name	
Group	
Date	

Password: E10BuJ@M

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "160001C" then the name of your source file will be "p1-160001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

Develop a Python program to read a list of positive integers as input from the keyboard and output whether the list contains duplicate numbers. Input numbers are separated by space and ends with a "-1". (This "-1" indicates the end of list and it is not one of the input numbers).

Your program must display the position of duplicate numbers in the list in sorted order. If there are n numbers in the list, then their positions are 1, 2, ..., n . You can assume that the maximum length of input list is 50. Print "None" if there are no duplicate values.

Format:

Input: A list of positive integers separated by a space, given in a single line. The list ends with -1.

Output: The positions of duplicate numbers in the list in sorted order (separated by a single space) or the string "None" exactly.

Sample:

Case 1

Input: 1 3 5 7 8 90 7 56 33 90 -1

Output: 4 6 7 10

Case 2

Input: 1 3 5 7 90 56 33 91 -1

Output: None

Case 3

Input: 5 5 5 5 1 5 5 -1

Output: 1 2 3 4 6 7

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDE or other text editor.
Example: If your Index No. is "180001C" then the name of your source file will be "p1-180001C.py"
3. If you use IDE, save your file in a folder named <Group name>_PP1_<Index number> in /Infs directory. For example, E3_PP1_180001C. The file must be uploaded to VPL.
4. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

In order to take a group photo, a group of students A, B, C, D, E, ... are initially standing in the order "A B C D E ...", ("A" is the leftmost) and waiting to get into the order of their heights. For this purpose, each student is given a card containing a letter and a value. Letter is either "L" or "R" which means LEFT or RIGHT. For e.g., if the card says "L2", then the student, from his current position, has to move to "his" LEFT, passing two students; it will be his final position. Students move one at a time, in the order A, B, C, D, E, ... Note that, after each student moves to a new position, relative positions of other students change. A student always starts moving from his current position.

Example: There are 6 students A, B, C, D, E, F and the cards given for them are L2, L4, L0, R2, R1, R2, respectively. Student A moves first; he has to move pass 2 students to the left side of him. After that, the students will be standing in the order, "B C A D E F". Then B has to move pass 4 students to the left side of him. After that, the student order will be "C A D E B F". Continuing this way, finally the students will be standing in the order "D C E F A B".

Develop a Python program to output the final position of each student, in the order of "A B C D E ...", as a number after they have changed their positions according to the given card. (In our Example above, A is standing in the 5th position. Therefore output is "5 6 2 1 3 4").

Format:

Input: Line 1: Student list

Line 2: List of cards containing letter L or R and a value, given to students (In the order of Line 1)

Output: Final position of each student as a number (in the order of Line 1 of input)

Sample:

Case 1

Input: A B C D E F
L2 L4 L0 R2 R1 R2
Output: 5 6 2 1 3 4

Case 2

Input: A B C D E
L0 R1 L2 R0 R1
Output: 2 1 5 4 3

Chocolate Cream Media

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals
2018 Intake, Jan - April 2019
Programming Problem 1
Time: 1 hour only
P2 [Wed 12.15]

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "180001C" then the name of your source file will be "p2-180001C.py"
3. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
4. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

A **prime number** is a positive integer such that its only divisors are 1 and the number itself. 2, 3, 5, 7, ... is the sequence of prime numbers. The n^{th} **Euclid number** is the product of first n prime numbers plus 1.

Example:

The 1st Euclid number is 3 because $3 = 2 + 1$

The 3rd Euclid number is 31 because $31 = 2 \times 3 \times 5 + 1$

The 4th Euclid number is 211 because $211 = 2 \times 3 \times 5 \times 7 + 1$

Develop a Python program to take an integer input n and output the n^{th} Euclid number. To decide if an integer p is prime, one method is to check for every $k \leq \text{square_root}(p)$ if k divides p . If no such k is found then p is prime.

Format:

Input: n

Output: n^{th} Euclid number

Sample:

Case 1

Input: 3
Output: 31

Case 2

Input: 6
Output: 30031

Explanation

For case 2, the 6th Euclid number is $2 \times 3 \times 5 \times 7 \times 11 \times 13 + 1$, which is 30031.

Develop a Python program to take a list (sequence) of positive integers as the input from the keyboard, find the most frequently occurring and the second-most frequently occurring numbers and display them. Reading of input numbers should stop when -1 appears in the input. You can assume that the maximum length of input list is 50 and the numbers in the input list are separated by space. You may handle unexpected inputs appropriately.

Note that there can be multiple numbers that can occur most frequently or second-most frequently. A special case is when each number appears only one time; in this case all the numbers occur most frequently and there are no second-most frequently occurring numbers.

Example execution session 1:

```
Enter the list: 5 3 2 4 8 9 7 1 6 5 7 5 -1
Most frequently occurring numbers = 5
Second most frequently occurring numbers = 7
```

Example execution session 2:

```
Enter the list: 1 2 3 4 5 6 7 8 9 10 -1
Most frequently occurring numbers = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Second most frequently occurring numbers =
```

Example execution session 3:

```
Enter the list: 5 3 2 4 8 9 7 1 6 5 4 7 5 7 -1
Most frequently occurring numbers = 5, 7
Second most frequently occurring numbers = 4
```

Example execution session 4:

```
Enter the sequence: 5 3 2 4 8 9 7 1 6 5 5 -1
Most frequently occurring numbers = 5
Second most frequently occurring numbers = 3, 2, 4, 8, 9, 7, 1, 6
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "160001C" then the name of your source file will be "p1-160001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

The first 5 digits of the 9-digit National Identity Card (NIC) number of Sri Lanka represents the date of birth and the gender of the owner. Develop a Python program that takes an NIC number as input and outputs the date of birth of the owner in the format yyyy/m/d.

Using digits 1-5 of the NIC number, here is how to find the year, month and date of birth of the person.

- Digits 1-2: represent the last two digits of year of birth, assumed to be in the range 1900-2000. (E.g., if the first 2 digits are "90", then the year of birth is 1990).
- Digits 3-5: represent the gender (male or female) and the day of the year on which the person's birth date falls. All years are assumed to have 29 days in February.
 - If male: the digits 3-5 is a number (the day in the year) in the range 001-366. If 001 the male is born on 1st of January. If 366, born on 31st of December. If 061, born on 1st of March.
 - If female: the digits 3-5 is a number in the range 501-866 (that is, the day in the year + 500). If 501 the female is born on 1st of January. If 866, born on 31st of December. If 561, born on 1st of March.

Format:

Input: A valid NIC number of a person born between 1900 and 2000. Format: 000000000V

Output: Date of birth in the format yyyy/m/d.

Sample:

Case 1

Input: 905014568V

Output: 1990/1/1

Case 2

Input: 800614128V

Output: 1980/3/1

Case 3

Input: 810614128V

Output: 1981/3/1

The Problem

Luhn formula is used to generate a check digit that is used to validate credit card numbers. Given an integer n digits $a_1, a_2, \dots, a_n$, the formula generates the check digit as follows:

1. Starting from the leftmost digit (a_1) and moving right, replace every second number with its double. If the result of doubling is greater than 9, then add the digits of the result. (E.g., if $a_2 = 6$, a_2 should be replaced with $2 \times 6 = 12$, which reduces with $1 + 2 = 3$.)
2. Take the sum of all the digits in the number from step 1.
3. Multiply the sum by 9. The check digit will be the remainder when this is divided by 10.

Develop a Python program that does the above with input and output as follows.

Format:

Input: A positive integer with one or more digits (note that n is unknown)

Output: A single integer denoting the check digit

Sample:

Case 1

Input: 987123

Output: 7

Explanation:

Number	9	8	7	1	2	3
Double every 2 nd digit	9	16	7	2	2	6
Modified number from step 1	9	7	7	2	2	6

Sum of digits = $9 + 7 + 7 + 2 + 2 + 6 = 33$

Check digit = $(33 * 9) \bmod 10 = 7$

Case 2

Input: 1234567

Output: 9



Description

Submission

Edit

Submission view

Lab 3- Additional Exercise 1

Requested files: Lab3_Ex1.py (Download)

Type of work: Individual work

Develop a program to implement a simple electricity charges calculator. First ask the user to enter number of units consumed. Then based on the number of units calculate the total bill of that user considering below rules and display the result.

- For first 50 units Rs. 0.50/unit
- For next 100 units Rs. 0.75/unit
- For next 100 units Rs. 1.20/unit
- For unit above 250 Rs. 1.50/unit
- An additional surcharge of 20% is added to all bills as taxes

Input : Input will be a single non-negative integer.

Output : Output should be a real number rounded to two decimal places.

Example

Case 1:

Input:

50

Output:

30.00

Case 2:

Input:

75

Output:

52.50

Index No.	
Name	
Group	
Date	

Password: E10AE#q9

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "160001C" then the name of your source file will be "p1-160001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

A *palindromic number* (or *palindrome*) is a number that remains the same when its digits are reversed (that is, it remains the same when read from left-to-right or right-to-left).

Examples: 1, 2, ..., 9, 11, 22, ..., 99, 111, ..., 484, 676, 10201, 12321.

You are given a number and you have to determine if it is a palindrome or not. If not, you should add the value of the number read from left-to-right and its value read from right-to-left, and check if the obtained sum is a palindrome. If not, you should continue the process, until you eventually obtain a palindrome.

Develop a Python program that takes an integer n ($1 \leq n < 1000$) as input and outputs the palindrome you can eventually obtain as described above. If the number of addition operations exceeds 25 you should output "Too long".

Format:

Input: A positive integer

Output: Palindrome obtained through the above process or the string "Too long" exactly.

Sample:**Case 1**

Input: 37

Output: 121

Explanation:

Since 37 is not a palindrome you will perform the addition: $37 + 73 = 110$. The number 110 is not a palindrome, either; so you will proceed to perform another sum: $110 + 011 = 110 + 11 = 121$. The obtained result is a palindrome, so you stop.

Case 2

Input: 7

Output: 7

Case 3

Input: 79

Output: 44044

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

In [options]
source
[destination]

Create a shortcut.

locate
filename

Search a copy of your
filesystem for the specified
filename.

lpr [options]

Send a print job.

ls [options]

List directory contents.

man
[command]

Display the help information
for the specified command.

mkdir
[options]
directory

Create a new directory.

mv [options]
source
destination

Rename or move file(s) or
directories.

passwd [name
[password]]

Change the password or allow
(for the system administrator)
to
change any password.

ps [options]

Display a snapshot of the

The Problem

Given a positive integer we are asked to generate a number sequence starting with the given integer and with the following rules. If the current number is 1 terminate the process. If the current number is even, consider the next number as $n/2$. If the current number is odd, consider the next number as $3*n+1$. The sequence will continue to grow until we get 1. For example if the given number is 22, the sequence will be {22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1}. The length of this sequence is 16. Develop a Python program to find the length of the sequence for a given positive integer.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The first line contains an integer N, the number of test cases. Each of the next N lines contains a positive integer for which a number sequence is to be generated.

Output: For each positive integer N, output the length of the generated sequence in a separate line.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	1	16
	22	
input_2.txt	2	112
	1000	1
	1	

- My home
- Site pages
- My profile
- Current course
 - In16-S1-CS1032 (104982)
 - Participants
 - Badges
 - General
 - 2 January - 8 January
 - 9 January - 15 January
 - 16 January - 22 January
 - 23 January - 29 January
 - 30 January - 5 February
 - 6 February - 12 February
 - 13 February - 19 February
 - Lecture 6 Slides
 - Sample Code Visualization for Functions
 - Coding Challenges (HackerRank)
 - Coding Challenges (TopCoder)
 - Submission for Lab 6
 - Lab 6- Additional Exercise 1**
 - Description
 - Submission
 - Edit
 - Submission view
 - Lab 6- Additional Exercise 2
- My courses

Administration

- Course administration
- My profile settings

Lab 6- Additional Exercise 1

Requested files: Lab6_Ex1.py (Download)

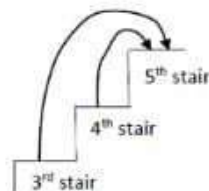
Type of work: Individual work

You are going to climb up a staircase that has an n number of stairs, starting from bottom. In each step, you can only move up either one or two stairs.

Note that, as an example, after reaching the 3rd or 4th stairs (see the figure below), you can climb up to the 5th stair in 2 ways;

I. move up 1 stair from the 4th stair, and

II. move up 2 stairs from the 3rd stair.



Develop a Python program to take an integer n as input ($0 < n < 120$) and display the number of ways you can climb up to the n^{th} stair. You may handle unexpected inputs appropriately.

Input: Single integer n ($0 < n < 120$)

Output: Single integer

Example

Case 1

Input:

5

Output:

8

Case 2

Input:

9

Output:

55

ps [options]	Display a snapshot of the currently running processes.
pwd	Display the pathname for the current directory.
rm [options] directory	Remove (delete) file(s) and/or directories.
rmdir [options] directory	Delete empty directories.
ssh [options] user@machine	Remotely log in to another Linux machine, over the network. Leave an ssh session by typing exit .
su [options] [user [arguments]]	Switch to another user account.
tail [options] [filename]	Display the last <i>n</i> lines of a file (the default is 10).
tar [options]	Store and extract files from a

- ▼ In16-S1-CS1032 (104982)
 - ▶ Participants
 - ▶ Badges
 - ▶ General
 - ▶ 2 January - 8 January
 - ▶ 9 January - 15 January
 - ▶ 16 January - 22 January
 - ▶ 23 January - 29 January
 - ▶ 30 January - 5 February
 - ▼ 6 February - 12 February
 - ▶ Lecture 5 Slides
 - ▶ Example Code Visualizations from Lecture 5
 - ▶ Submission for Lab 5
 - ▶ Lab 5- Additional Exercise 1
 - ▶ Lab 5- Additional Exercise 2
 - Description
 - Submission
 - Edit
 - Submission view
 - ▶ 13 February - 19 February
- ▶ My courses

Administration

- ▶ Course administration
- ▶ My profile settings

Requested files: Lab5_Ex2.py (Download)

Type of work: Individual work

The transpose of a matrix is a new matrix whose rows are the columns of the original.

Write a program to produce the transpose of a given matrix using loops.

Input: The first two integers will represent the number of rows (R) and number of columns (C) respectively. The R lines that follows will have elements in each row separated by spaces.

Enter No of rows = R

Enter No of columns = C

$e_{1,1}$ $e_{1,2}$.. $e_{1,C}$

$e_{2,1}$ $e_{2,2}$.. $e_{2,C}$

.

$e_{R,1}$ $e_{R,2}$.. $e_{R,C}$

Output: Display the transposed matrix. Each row should start in a new line and elements in a row will be separated by commas.

Example

Input:

Enter No of rows = 2

Enter No of columns = 3

11 12 13

21 22 23

Output:

11 21

12 22

13 23

The Problem

Given a positive integer we are asked to generate a number sequence starting with the given integer and with the following rules. If the current number is 1 terminate the process. If the current number is even, consider the next number as $n/2$. If the current number is odd, consider the next number as $3*n+1$. The sequence will continue to grow until we get 1. For example if the given number is 22, the sequence will be {22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1}. The length of this sequence is 16. Develop a Python program to find the length of the sequence for a given positive integer.

- The input is a text file in the same folder. You get the text file name as input from the terminal to your program.
- Have a function named "getText" to read input from file. Handle invalid input properly and output the string 'INVALID' to file and display a meaningful message on screen.
- Have a function named "showResult" to display the output on the screen and to write it to a file named "result.txt" in the same folder. You may define additional functions in your code.
- The main program should call the functions appropriately to achieve the task.

Format:

Input (via terminal): Name of the input file in a single line.

Input File: The first line contains an integer N, the number of test cases. Each of the next N lines contains a positive integer for which a number sequence is to be generated.

Output: For each positive integer N, output the length of the generated sequence in a separate line.

Sample:

Input Filename	Input File	Output (File and on screen)
input_1.txt	1 22	16
input_2.txt	2 1000 1	112 1

Instructions:

1. Name your source file as "p1-<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
 2. Upload the source file, "p1-<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "150001C" then the name of your source file will be "p1-150001C.py"

The Problem

In number theory, a *deficient* (or *deficient number*) is defined as an integer n for which the sum of divisors $< 2n$. The value $(2n - \sum(\text{divisors}))$ is called the number's *deficiency*. In contrast, an *abundant number* can be defined as an integer n for which the sum of its divisors $> 2n$. The value $(\sum(\text{divisors}) - 2n)$ is the number's *abundance*.

Example 1: Consider the number 21. Its divisors are 1, 3, 7 and 21, and their sum is 32. Because 32 is less than 2×21 , the number 21 is deficient. Its deficiency is $2 \times 21 - 32 = 10$.

Example 2: The integer 12 is the first abundant number. Its divisors are 1, 2, 3, 4, 6, and 12, and their sum is 28. Because 28 is greater than 2×12 , the number 12 is abundant. Its abundance is $28 - 24 = 4$.

If a number is neither deficient nor abundant, then it is a *perfect number*, whose sum of divisors $= 2n$.

Develop a Python program to take as input an integer n ($0 < n < 100000$) and output whether it is "abundant" or "deficient" (followed by how much) or "perfect", as shown in the examples below. You may handle unexpected inputs appropriately.

Example executions are shown below for $n=12, 6, 21, 10000, 999$:

Input n: 12
12 is abundant by 4

Input n: 6
6 is perfect

Input n: 21
21 is deficient by 10

Input n: 10000
10000 abundant by 4211

Input n: 999
999 is deficient by 478

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Let us define the **distance** between two numbers as the distance between their binary (base 2) representations. The **distance between two binary numbers of equal length is the number of positions at which the corresponding bits are different**. For example, the distance between the two 4-bit numbers $b_1=1001$ and $b_2=1110$ is 3 because the corresponding bits differ at 3 positions: the 0th (LSB), 1st and 2nd bit positions in b_1 and b_2 ; the bits in the 3rd bit position (MSB) in b_1 and b_2 are the same. As another example, distance between 101101 and 1001001 is 2.

Develop a Python program that:

- reads as inputs 3 positive decimal integers m , n and p ($0 < m < n$, $m < n < 1000$, $m < p < n$, $(n-m) > 10$), and
- outputs decimal integers X and Y where $m \leq X \leq n$, $X \neq p$ and X has the smallest distance from p and $m \leq Y \leq n$, $Y \neq p$ and Y has the largest distance from p .

Note that there could be more than one X and/or more than one Y in a given situation. When multiple X or Y exist, their ordering in the output is not important. You may assume 32-bit representation for numbers. You may handle unexpected inputs appropriately.

Example execution session 1:

```
Enter m: 2
Enter n: 14
Enter p: 10
Answer X: 8, 2, 11, 14
Answer Y: 5
```

Example execution session 1:

```
Enter m: 129
Enter n: 142
Enter p: 135
Answer X: 134, 133, 131
Answer Y: 136
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff]

CS1032 Programming Fundamentals
2015 Batch, Jan - May 2016
Programming Assignment 1
Time: 1 hour only
Q17 [C4, Tue 1.15]

Registration No: 151038D
Name: C.S.H. Thiruvananthapuram
Group: CH
Date: 2

Instructions:

1. Name your source file as "p1-<StudentNum>.py" and save it in a folder/directory named "p" your home directory.
2. Upload the source file, "p1-<StudentNum>.py" onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end. Make sure you name your source file correctly. Example: If your student number is "150001C" then the name of your source file will be "p1-150001C.py"

The Problem

There are n number of students in a class. For the year end concert, the teacher wants to pair the students by their heights for a special dance. If the pair of students are of same height, they can participate for the dance. If a student does not have a partner with matching height, he/she is assigned to the support crew. The teacher has a list of heights and wants to know how many pairs will be participating for the dance this year.

Develop a Python program to read a list of positive integers (the list of heights) separated by space input from the keyboard and output the number of dancing pairs from the class. Each integer n satisfies $100 \leq n \leq 200$. You may handle unexpected inputs appropriately.

Example execution session 1:

Enter the heights: 135 130 135 150 140 135
 0 1 2 3 4 5
Number of Dancing Pairs: 1

Example execution session 2:

Enter the heights: 135 130 145 135 140 145 150 155
Number of Dancing Pairs: 2

Example execution session 3:

Enter the heights: 135 130 135 135 140 135 135
Number of Dancing Pairs: 2

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to consider as non submission of the assignment. Do not write anything else unless asked]

CS1032 Programming Fundamentals

2015 Batch, Jan - May 2016

Programming Assignment 1

Time: 1 hour only

Q16 [C4, Tue 1.15]

Registration No:

Name:

Group:

Date:

Instructions:

1. Name your source file as "p1-<StudentNum>.py" and save it in a folder/directory named "p1" in your *home* directory.
 2. Upload the source file, "p1-<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "150001C" then the name of your source file will be "p1-150001C.py"

The Problem

Two strings are *anagrams* if they are written using the exact same letters, ignoring space, punctuation and capitalization. Each letter should have the same count in both strings.

For example Army and Mary are anagrams of each other.

Develop a Python program to take two strings as input and output whether they are anagrams. An input string can have multiple words separated by space. Only alphabetical characters are allowed in a word. You may handle unexpected inputs appropriately.

Example execution session 1:

Enter string one: Eleven plus two

Enter string two: Twelve plus one

These are anagrams

Example execution session 2:

Enter string one: Eleven plus four

Enter string two: Fourteen plus one

These are not anagrams

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Group A4 – PA1's Question 17/18 - Not included!

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q7 [A1 –1]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

Develop a Python program that takes a sequence of integers as input and outputs the number of powers of 2 in that sequence. A power of 2 is 2^n where n is a positive integer.

Example execution session:

Input: 23 32 1 45 67 128 2048 1024 87 16

Number of powers of two: 6

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q8 [A1 –2]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

Develop a Python program to take as input 3 words and to check whether the first two words exist within the third word. If either of the first 2 words exists in the third word, not as a complete word, but the letters of each word is in order but scrambled within the third word, then output should be given as SHUFFLE. Otherwise if both of the first two words exists in the third word, as the complete word itself, then out should be given as NOT A SHUFFLE.

Example execution session 1:

Enter word 1: joy
Enter word 2: enable
Enter word 3: ksldotiyejnshaqwbolpe
Output: SHUFFLE

Example execution session 2:

Enter word 1: joy
Enter word 2: enable
Enter word 3: pijoydkfenables
Output: NOT A SHUFFLE

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q23 [A2 –1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “150001C” then the name of your source file will be “p1-150001C.py”

The Problem

There are p players in a tennis tournament in which two players play a tennis match against each other and one will win. The matches are played over several *rounds*, which are numbered 1, 2, 3 Initially, all p players enter round 1. The rounds will proceed as follows.

- Let i the current round, let n be the number of the players entering round i ($n = p$ when $i=1$)
- Let $m = 2^k$ for some integer $k>0$, and m be the largest possible such that $m \leq n$.
- m players will play $m/2$ matches in round i ; the $m/2$ winners will enter round $i+1$; the $m/2$ losers will not play any more matches (they will not enter round $i+1$)
- $(n - m)$ players who enter round i will not play any match in round i and enter round $i+1$ directly

- The tournament ends after the round in which $n=2$; there is only 1 match (the final match) and the winner in that is the winner of the tournament

Develop a Python program that takes as input p , where p ($2 \leq p \leq 1000$) is the number of players entering the tournament, and outputs the number of rounds and the total number of matches played until a winner is selected. You may handle unexpected inputs appropriately.

Example execution session 1:

Number of players, p : 12

Number of rounds=4, matches=11

Explanation for case $p=12$

Player	Round 1	Round 2	Round 3	Round 4
1	Match 1	Match 7	Match 9	Match 11 (final)
2				
3	Match 2			
4				
5	Match 3	Match 8		
6				
7	Match 4			
8				
9		Match 5	Match 10	
10				
11		Match 6		
12				

Tournament has 4 rounds when $p=12$ (table above shows one possible scenarios; there can be other valid scenarios)

Example execution session 2:

Number of players, p : 29

Number of rounds=7, matches=28

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q24 [A2 –2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “150001C” then the name of your source file will be “p1-150001C.py”

The Problem

To identify companies whose stocks are worth buying, Kamal, a stock broker has come up with a new idea. If a company on a specific day has a stock price which is greater than $(10 + \text{the average stock price})$

of that week), such days are called “valued days”. If a company has 3 valued days in a week, Kamal recommends buying stocks of that company. Otherwise he recommends not to buy.

Example:

Suppose the daily stock prices of a company over a week are as follows:

70.5 78.9 90.7 33.2 56.0 40.2 22.1

The average stock price for the week is 55.9. There are 3 “valued days” (days 1, 2 and 3) on which the stock price is greater than (10 + average). Therefore Kamal will recommend to buy.

Develop a Python program that takes as input the daily stock prices over a week separated by space and outputs the number of “valued days” and the recommendation whether to buy or not to buy stocks (output as either “RECOMMENDED” or “NOT RECOMMENDED”). Each input (daily stock price) is a positive number, possibly a non-integer. You may handle unexpected inputs appropriately.

Example execution session 1:

Input stock prices of the week: 70.5 78.9 90.7 33.2 56.0 40.2 22.1

Number of valued days: 3

RECOMMENDED

Example execution session 2:

Input stock prices of the week: 33.5 46.9 40.7 43.2 26.0 30.2 32.1

Number of valued days: 1

NOT RECOMMENDED

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q19 [A3 –1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

A *garland* word is a word that starts and ends with the same N letters in the same order, for some N where $0 < N < (\text{length of the word})$. The maximum N that maintains this property is the *garland word's degree*. For example, "onion" is a garland word of degree 2, because the first two letters "on" are the same as its last letters. The name "garland word" comes from the fact that you can make chains of words as follows.

```
onionionionionionionionionionionionionionionionion...
```

Develop a Python program that takes as input a word and outputs the degree of the word if it is a garland word. And 0 otherwise. An input word can have only alphabetical characters. Lower-case- Upper case differences between letters must be ignored; that is, for example, “a” is considered the same as “A”. You may handle unexpected inputs appropriately. You can use the following functions if needed.

Function	Description	Example
max()	Returns the larger between two arguments	max(3, 5) returns 5
isalpha()	Return if the character is alphabetic	'c'.isalpha() returns True '%'.isalpha() returns False
lower()	Converts a character to lowercase	'C'.lower() returns 'c'
upper()	Converts a character to uppercase	'a'.upper() returns 'A'

Example execution sessions are shown below:

```
Enter the input word: PrograMMer
Degree = 0
```

```
Enter the input word: CeraMiC
Degree = 1
```

```
Enter the input word: alfalfa
Degree = 4
```

```
Enter the input word: onion
Degree = 2
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals
2015 Batch, Feb – June 2016
Programming Assignment 1
Time: 1 hour only
Q20 [A3 –2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.

2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

You are going to balance a word at one of its letters called the *balance point*. The parts of the word that are on the left and right of the balance point are the *left-part and right-part*, respectively. A word can be balanced if a balance point exists such that the weights of left and right parts are equal. But some words cannot be balanced. The weight of a part of the word is the sum of the weights of its letters. The weight of a letter = (its position in the English alphabet) x (its distance from the balance point). The positions are A=1, B=2, C=3,, Z=26 (we ignore whether it is lower or upper case).

Example: Consider the word "STeAd". This word balances at "T", as shown below.

Left-part = "S"; weight of left-part = weight of "S" = position in alphabet x distance = 10 x 1 = 19

Right-part = "EaD"; weight of right-part = weight of "E" + weight of "a" + weight of "D"

$$= (5 \times 1) + (1 \times 2) + (4 \times 3) = 19$$

Therefore T is the balance point and that the weight of each part is 19.

Develop a python program that takes as input a word, determines if it can be balanced, and, if yes, then outputs the balance point and the weight of one part, otherwise outputs "Cannot be balanced". Input must have only letters (alphabetical characters) and can be either lower or upper case. You may handle unexpected inputs appropriately. You can use the following function if needed.

Function	Description	Example
isalpha()	Return if the character is alphabetic	'c'.isalpha() returns True '%.isalpha() returns False
lower()	Converts a character to lowercase	'C'.lower() returns 'c'
upper()	Converts a character to uppercase	'a'.upper() returns 'A'

Example execution sessions are shown below:

Enter the input word: ConsubSTantiation
 Balance point letter: "a", weight=456

Enter the input word: Superglue
 Cannot be balanced!

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

Programming Assignment 1

Time: 1 hour only

Q1 [B1 – 1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “150001C” then the name of your source file will be “p1-150001C.py”

The Problem

Swedish children have developed a secret language that only kids know, so they can hide secrets from their confused parents. This language is known as “Rövarspråket” (means “Robber’s language”).

This secret language is not very complicated, each ordinary English word is code by replacing each consonant with the consonant doubled (repeated) and with an “o” in between. If the consonant is in uppercase, the repeating consonant is in lowercase. So the consonant “b” is replaced “bob”, “r” is replaced by “ror”, “n” is replaced with “non”, “G” is replaced by “Gog” and so on. Vowels and other non-alphabetic characters are left intact (not changed).

Example:

Input:	Hello World!
Output (Encoded):	Hohelollolo Wowororloldod!

Input:	b
Output (Encoded):	bob!

Develop a Python program to encode a given input sentence into “Rövarspråket” and outputs the encoded string. You may handle unexpected errors appropriately.

You may use the `isalpha()` Python function to check whether a given character is alphabetic.

Examples: `"C".isalpha() => True` `"%".isalpha() => False`

You may use the `lower()` Python function to convert a character to lowercase and `upper()` Python function to convert a character to uppercase.

Examples: `"C".lower() => 'c'` `"a".upper() => 'A'`

Example execution session:

Input: Hello World!

Output: Hohelollolo Wowororloldod!

Input: where is god!

Output: wowhoherore isos gogodod!

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q2 [B1 – 2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

A *palindromic number* (or *palindrome*) is a number that remains the same when its digits are reversed (that is, it reads the same from left-to-right and right-to-left)

Examples: 1, 2,, 9, 11, 22,, 99, 111,, 484, 676, 10201, 12321

A double-base palindrome is a palindromic number that maintains palindromic property in two bases.

Examples: $99_{10} = 1100011_2$

$585_{10} = 1001001001_2$

Develop a python program that repeatedly takes an integer ($1 \leq n \leq 100000$) as input and outputs whether the given number is a **Double Base palindrome** in base 10 and base 2. You have to continue to take inputs and repeat this, until the user enters -1 as the input. You may handle unexpected inputs appropriately.

You may use the Python built-in function “bin()” in your program to obtain the binary (base 2) representation of a number. Example: `bin(23) = '0b10111'`

Sample execution of the program should be similar to the following.

Enter Input: 99

Yes

Enter Input: 585

Yes

Enter Input: 100

No

Enter Input: 1222

No

Enter Input: 56982

No

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q3 [B2 – 1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

Develop a python program that takes as input a word (consisting only of letter from the alphabet) and output if the letters in the input word are in alphabetical order (IN ORDER), reverse alphabetical order (IN REVERSE ORDER) or neither (NOT IN ORDER). Each letter can be either lower or upper case but this must be ignored. You may use the following functions if you need.

Function	Description	Example
ord()	Returns the decimal value of the ASCII code	ord('a') returns 97
isalpha()	Return if the character is alphabetic	'c'.isalpha() returns True '%.isaplha() returns False
lower()	Converts a character to lowercase	'C'.lower() returns 'c'
upper()	Converts a character to uppercase	'a'.upper() returns 'A'

Example execution session 1:

Enter word: aaaaa

Output: aaaaa IN ORDER

Example execution session 2:

Enter word: alMost

Output: alMost IN ORDER

Example execution session 3:

Enter word: Wronged

Output: Wronged IN REVERSE ORDER

Example execution session 4:

Enter word: random

Output: random NOT IN ORDER

Example execution session 5:

Enter word: toFfeE

Output: toFfeE IN REVERSE ORDER

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q4 [B2 – 2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

“Disemvoweling” means removing the vowels and spaces between words from a given text. The letters a,e,i,o,u are vowels. The idea is to make the text difficult but not impossible to read. We will keep the vowels we removed (in their original order) as well.

Example:

Original string (INPUT): two drums and a cymbal fall off a cliff

Disemvoweled string: twdrmsndcymbflflffclff

Removed vowels: ouaaaaoai (In original order)

Develop a python program that takes as input a string of words separated by space, disemvowel it and print disemvoweled string and removed vowels (in order of appearance in the input). Input can contain only alphabetic characters (upper and lower case) and space. You have to maintain the case of each letter in the output.

Example execution session 1:

Enter the input: Hello World

Disemvoweled: HllWrld

Vowels: eoo

Example execution session 2:

Enter the input: two drums and a cymbal fall off a cliff
Disemvoweled: twdrmsndcymbflflffclff
Vowels: ouaaaaoai

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q5 [B3 – 1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

A gaming company tracks the number of games sold, when a customer buys a game, the number of games sold is increased by one. At that point, if the number of games sold, **is not** divided by any number between 2 and 10, the developer gets a reward.

Develop a python program that takes an integer n , ($1 \leq n \leq 10^4$), and output the number of rewards.

When inputted 29, the rewards are given at 1, 11, 13, 17, 19, 23 and 29. So the number of rewards are 7.

Example execution 1:

Enter Input: 29

Number of rewards: 7

Example execution 2:

Enter Input: 12

Number of rewards: 2

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q6 [B3 –2]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

Develop a Python program which takes any positive integer n ; ($10 < n < 10,000$) as an input and print 0 with n if the number divisible by 3. If input n is not divide by 3, then print 1 with n if the consecutive integer $n+1$ is divisible by 3 or print -1 with n if the consecutive integer $n-1$ is divisible by 3. This process is continued for the each quotient obtained after division by 3 until $n=1$ is obtained.

Example execution 1:

Enter Input: 998

Output:

```
998 - 1
333 - 0
111 - 0
 37 - -1
 12 - 0
  4 - -1
  1
```

Example execution 2:

Enter Input: 243

Output:

```
243 - 0
 81 - 0
 27 - 0
  9 - 0
  3 - 0
  1
```

Example execution 3:

Enter Input: 135

Output:

```
135 - 0
 45 - 0
 15 - 0
  5 - 1
  2 - 1
  1
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q13 [B4 –1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

Jimith entered into a calculator the final answer for a homework problem, left the calculator on the table and went to sleep. Meanwhile his little brother, Arosh found the calculator and started to press keys randomly. Now the number displayed on the calculator is a string N of digits with two parts: the “prefix” (P) which is the substring of digits first entered by Jimith and the “suffix” (S), which is the substring of random digits later entered by Arosh. (i.e. $N = PS$). After waking up, Jimith got to know that Arosh had pressed at least one random key. Unfortunately Jimith has forgotten P, the number he himself entered: he only remembers that it was divisible by 4. Now he wants to find out P.

Develop a Python program to take as input N, the final string of digits on the calculator display, and output possible prefixes P which are divisible by 4. A substring can start with zero. Note that the $3 \leq \text{length of } N \leq 16$ and S has at least one digit. You may handle unexpected inputs appropriately.

Example: If $N = \text{“447295”}$, then there are 3 possible prefixes P divisible by 4: 4472, 44 and 4.

Example execution session1:

Number displayed N: 124

Possible prefixes: 12

Example execution session 2:

Number displayed N: 447295

Possible prefixes: 4472, 44, 4

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q14 [B4 –2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “150001C” then the name of your source file will be “p1-150001C.py”

The Problem

The life goes up and down, just like nice sequences. A sequence $t_1, t_2, \dots, t_n$ is called a nice sequence following two conditions are satisfied:

- $t_i < t_{i+1}$ for each odd $i < n$
- $t_i > t_{i+1}$ for each even $i < n$

Otherwise, the sequence is not nice.

3 example nice sequences: (2, 8), (1, 5, 1) and (2, 5, 1, 100, 99, 120)

3 example not nice sequences: (1, 1), (1, 2, 3) and (2, 5, 3, 2)

Develop a Python program that takes as input a sequence of positive integers and outputs whether it is nice or not. Input integers are separated by space. A sequence has at least 2 integers and at most 9 integers. You may handle unexpected inputs appropriately.

Example execution session 1:

Enter the sequence: 2 5 1 100 99 120

Result: nice

Example execution session 2:

Enter the sequence: 2 5 3 2

Result: not nice

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q15 [C1 –1]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

In number theory, a deficient is defined as an integer n for which the sum of divisors $< 2n$. The value $2n - \sum(\text{divisors})$ is called as the number's deficiency. In contrast, an abundant number can be defined as an integer n for which the sum of divisors $> 2n$. The value $\sum(\text{divisors}) - 2n$ is the numbers' abundance.

Example 01: Consider 21. Its divisors are 1, 3, 7 and 21 and their sum is 32. Because 32 is less than 2×21 , the number is deficient. Its deficiency value is $2 \times 21 - 32 = 10$.

Example 02: Consider 12, its divisors are 1,2,3,4,5,12 and their sum is 28. Because 28 is greater than 2×12 , the number is abundant. Its abundant value is $28 - 2 \times 12 = 4$.

If number is neither deficient nor abundant, it is called as a perfect number whose sum of divisors = $2n$.

Develop a python program that repeatedly takes an integer ($1 \leq n \leq 100000$) as input and outputs whether the given number is deficient and its deficiency, abundant and its abundance or whether a number is perfect.

Example execution session:

```
Input n: 12
12 is abundant by 4
Input n: 6
6 is perfect
Input n: 21
21 is deficient by 10
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q16 [C1 –2]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.

2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

Develop a python program that repeatedly takes as input a non-negative integer n , which consists of at least 2 and at most 100 digits. Then it should do the following.

- If n is divisible by 8, then output is "Yes".
- If n is not divisible by 8, then try to obtain m , the largest number divisible by 8 by removing a minimum number of consecutive digits from n (but re-arranging digits is not allowed) and do one of the following:
 - If such a number m exists, then output "Yes" and the value of m .
 - If such a number m does not exist, then output "No".

Example execution session are shown below for $n=3454$, 96, 10, 11111, and 964:

Input: 3454
Yes m=344

Input: 96
Yes

Input: 10
No

Input: 11111
No

Input: 964
Yes m=96

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

**CS1032 Programming Fundamentals
2015 Batch, Feb – June 2016
Programming Assignment 1
Time: 1 hour only
Q11 [C2 –1]**

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.

2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

We can write the cube of any given integer n as the sum of n consecutive odd integers.

Examples: $1^3 = 1 = 1$
 $2^3 = 8 = 3 + 5$
 $3^3 = 27 = 7 + 9 + 11$
 $5^3 = 125 = 21 + 23 + 25 + 27 + 29$

Develop a Python program to take an integer n as input ($1 \leq n \leq 20$) and output the set of n consecutive odd numbers whose summation is equal to n^3 . The program should repeat this process with the next input, until the input is -1. You may handle unexpected inputs appropriately.

Example execution session:

Enter Input: 5
Odd Number Sequence: 21 + 23 + 25 + 27 + 29
Enter Input: 8
Odd Number Sequence: 57 + 59 + 61 + 63 + 65 + 67 + 69 + 71
Enter Input: -1
Bye!

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals
2015 Batch, Feb – June 2016
Programming Assignment 1
Time: 1 hour only
Q12 [C2 –2]

Instructions:

1. Name your source file as "p1<StudentNum>.py" and save it in a folder/directory named "p1" in your home directory.
 2. Upload the source file, "p1<StudentNum>.py" onto the relevant area on Moodle.
 3. Write your name and other details in the above box and return this sheet to staff at the end.
- Make sure you name your source file correctly. Example: If your student number is "**150001C**" then the name of your source file will be "**p1-150001C.py**"

The Problem

A palindromic word (or palindrome) is a string that remains the same when its letters are reversed (that is, it remains the same when read left-to-right or right-to-left)

Examples: madam, civic, reviver, rotor, refer, level, noon.

Develop a python program to take as input a word and output whether the input word is palindrome or not. If not the program should identify the longest palindromic substring (whose length is at least 9) in the input word. For example even though “afternoon” is not a palindrome “noon” is a palindrome. If no such substring is found, output “There are no palindromes”. Then the program should repeat this by taking the next input word and continuing in the same manner. The program will stop when the input word is “bye”.

Note that the input must be a single word. You may handle unexpected inputs appropriately.

Example execution session:

```
Enter word: reference
Longest Palindrome: refer
Enter word: level
Longest Palindrome: level
Enter word: mouse
There are no palindromes
Enter word: Bye
Bye!
```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

**CS1032 Programming Fundamentals
2015 Batch, Feb – June 2016
Programming Assignment 1
Time: 1 hour only
Q21 [C3 –1]**

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

Swedish children have developed a secret language that only kids know, so they can hide secrets from their confused parents. This language is known as “Rövarspråket” (means “Robber’s language”).

This secret language is not very complicated, each ordinary English word is code by replacing each consonant with the consonant doubled (repeated) and with an “o” in between. If the consonant is in uppercase, the repeating consonant is in lowercase. So the consonant “b” is replaced “bob”, “r” is replaced by “ror”, “n” is replaced with “non”, “G” is replaced by “Gog” and so on. Vowels and other non-alphabetic characters are left intact (not changed).

Example:

Input:	Hello World!
Output (Encoded):	Hohelollolo Wowororloldod!

Input:	b
Output (Encoded):	bob!

Develop a Python program that Swedish parents can use to decode what their kids are saying. The input string can have a maximum length of 200. You may handle unexpected errors appropriately.

You may use the `isalpha()` Python function to check whether a given character is alphabetic.

Examples: `"C".isalpha() => True` `"%".isalpha() => False`

You may use the `lower()` Python function to convert a character to lowercase and `upper()` Python function to convert a character to uppercase.

Examples: `"C".lower() => 'c'` `"a".upper() => 'A'`

Example execution session:

Input: Hohelollolo Wowororloldod!

Output: Hello World!

Input: wowhoherore isos gogodod!

Output: where is god!

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q22 [C3 –2]

Instructions:

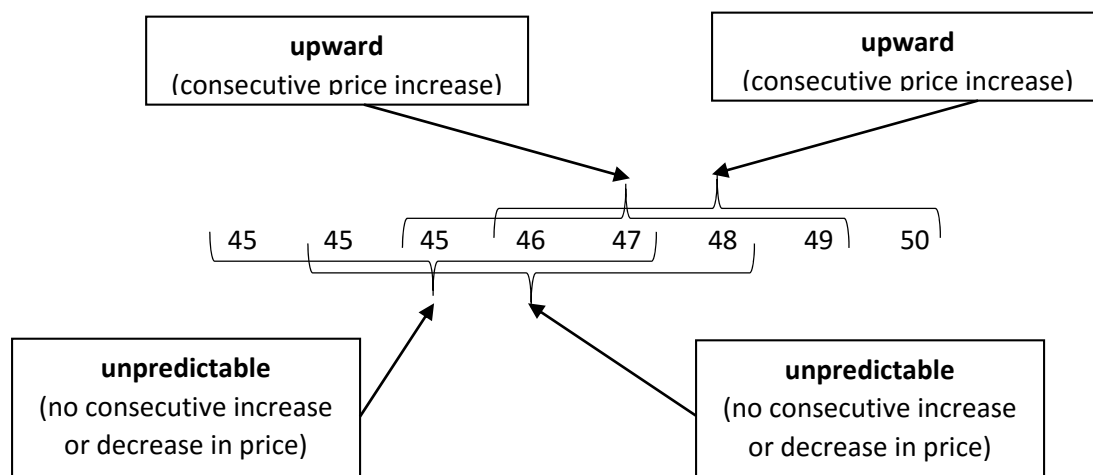
1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “150001C” then the name of your source file will be “p1-150001C.py”

The Problem

The Finance Ministry of the country Holland found, by research that the economic trend of the country can be roughly identified by the price of bread. If the regulated price of bread increases for 5 consecutive weeks, the economic trend is **upward**. If the price decreases for five consecutive weeks, then the economic trend is **downward**. If there is no such pattern in bread prices, the economic trend is **unpredictable**.

Example: Price of bread over 8 weeks



In the above example, within the 8 week sequence, there are four 5 week windows. As can be seen the first two windows have an unpredictable trend and the last two windows have an upward trend.

Develop a python program that takes as input a sequence of weekly bread prices separated by space ($5 \leq \text{number of weeks} \leq 20$) and outputs the economic trend of each 5 week window. Each bread price is a positive integer. You may handle unexpected inputs appropriately.

Example execution session 1:

Bread Prices: 45 45 45 46 47 48 49 50

Trend: unpredictable, unpredictable, upward, upward

Example execution session 2:

Bread Prices: 45 44 43 42 41 44

Trend: downward, unpredictable

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q9 [C4 –1]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

Two strings are anagrams if they are written using the exact same letters, ignoring space, punctuation and capitalization. Each letter should have the same count in both strings.

For example Army and Mary are anagrams of each other.

Develop a Python program to take two strings as inputs and out whether they are anagrams. An input string can have multiple words separated by space. Only alphabetical characters are allowed in a word. You may handle unexpected inputs appropriately.

Example execution session 1:

Enter string one: Eleven plus two

Enter string two: Twelve plus one

These are anagrams

Example execution session 2:

Enter string one: Eleven plus four

Enter string two: Fourteen plus one

These are not anagrams

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2015 Batch, Feb – June 2016

Programming Assignment 1

Time: 1 hour only

Q10 [C4 –2]

Instructions:

1. Name your source file as “p1<StudentNum>.py” and save it in a folder/directory named “p1” in your home directory.
2. Upload the source file, “p1<StudentNum>.py” onto the relevant area on Moodle.
3. Write your name and other details in the above box and return this sheet to staff at the end.

Make sure you name your source file correctly. Example: If your student number is “**150001C**” then the name of your source file will be “**p1-150001C.py**”

The Problem

There are n number of students in a class. For the year end concert, the teacher wants to pair the students by their heights for a special dance. If the pair of students are of the same height, they can participate for the dance. If a student does not have a partner with matching height, he/she is assigned to the support crew. The teacher has a list of heights and wants to know how many pairs will be participating for the dance this year.

Develop a Python program to read a list of positive integers (the list of heights) separated by spaces as the input from the keyboard and output the number of dancing pairs from the class. Each integer n satisfies $100 \leq n \leq 200$. You may handle unexpected inputs appropriately.

Example execution session 1:

Enter the heights: 135 130 135 150 140 135
Number of dancing pairs: 1

Example execution session 2:

Enter the heights: 135 130 145 135 140 145 150 155
Number of Dancing Pairs: 2

Example execution session 3:

Enter the heights: 135 130 135 135 140 135 135
Number of Dancing Pairs: 2

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non-submission of the assignment. Do not write anything else unless asked by staff.]

Index No.	Denny C
Name	
Group	E11
Date	

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<indexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py".
3. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
4. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

The birthday and gender (sex) can be derived using the National Identity Card (NIC) number of a Sri Lankan citizen. We will consider the 9-digit NIC numbers where the first 5 digits represent the date of birth. For example,

901912190V

The first 2 digits represent the last two digits of birth year, which must be between 1900 and 1999. In the above example, as the first 2 digits are "90", the birth year is 1990. For males, the next 3 digits represent the number of days to the birthday from January 1. For females, these 3 digits starts from 500. Hence, for females, the next 3 digits give 500 + numbers of days to the birthday from January 1. The month and day of the date of birth and the gender of a person can be derived from this information.

You can use the following information when deriving the month and day:

January - 31 days, February - 29 days, March - 31 days, April - 30 days, May - 31 days, June - 30 days, July - 31 days, August - 31 days, September - 30 days, October - 31 days, November - 30 days and December - 31 days.

Note: When deriving the month and day from the 3rd, 4th and 5th digits, always consider the month of February to have 29 days irrespective of the birth year being a leap year or not.

Develop a Python program which can output a person's birthday and gender when his/her NIC is given as input, as follows.

Format:

Input: The 9-digit NIC number of a person (without the final character 'V')

Output: The birthday in the format yyyy-mm-dd followed by 'M' or 'F' ('M' for male and 'F' for female)

Sample:**Case 1**

Input: 901912190
Output: 1990-07-09
M

Case 2

Input: 928001432
Output: 1992-10-26
F

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
3. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL
4. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

The **Grade Point Average** (GPA) of a student can be calculated using the following formula, where n is the number of subjects, p_i and w_i are the grade point and weight of subject i .

$$GPA = \frac{\sum_{i=1}^n p_i w_i}{\sum_{i=1}^n w_i} = \frac{p_1 w_1 + p_2 w_2 + p_3 w_3 + \dots + p_n w_n}{w_1 + w_2 + w_3 + \dots + w_n}$$

The **average grade** of a student is a letter grade (e.g., A, B, C,...) indicating the average performance across all the subjects; it is obtained by rounding his/her GPA to the nearest grade point in the **grading schema**. The grading schema defines the grade point of each letter grade.

Develop a Python program to output the GPA and the average grade of a student when the letter grades obtained for n subjects ($n > 1$), the weights for each subject and the grading schema are given as input.

Format:

Input: line 1: Number of subjects (n) and number of rows (m) in the grading schema, separated by space
 line 2: Letter grades for the n subjects (letter grades should be in the grading schema)
 line 3: Weights of the n subjects
 line 4 - line (4+ m -1): Grading schema in m lines, each containing a letter grade and its grade point

Output: GPA of the student followed by the average grade. GPA should be rounded to two decimal places

Sample:**Case 1**

Input:

```

5 4
A B B D C
3 2 2 1 3
A 4.0
B 3.0
C 2.0
D 0.0

```

Output:

```

2.73
B

```

Case 2

Input:

```

3 4
B C C
1 3 2
A 4.2
B 3.7
C 2.5
D 0.0

```

Output:

```

2.70
C

```

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Index No.	
Name	
Group	
Date	

Instructions:

1. Use the given password to access the relevant Virtual Programming Lab on Moodle.
2. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
 Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
3. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
4. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

Collatz conjecture is a positive integer series $a_0, \dots, a_i$ where, whatever the initial value $a_0 = n$, $n > 0$, the last value a_i will always be 1. Given the initial value $a_0 = n$, the smallest i such that $a_i = 1$ is called the **Total Stopping Time** of n . The sequence can be calculated as follows:

$$a_i = \begin{cases} \frac{1}{2} a_{i-1} & \text{if } a_{i-1} \text{ is even} \\ 3a_{i-1} + 1 & \text{if } a_{i-1} \text{ is odd} \end{cases}$$

Example: The following table includes Collatz conjectures for initial values $n = 1, 2, 3, 6$ and, for each case, the **Total Stopping Time** and the **Maximum value** in the series.

n	Series	Total Stopping Time	Maximum Value
1	1	0	1
2	2, 1	1	2
3	3, 10, 5, 16, 8, 4, 2, 1	7	16
6	6, 3, 10, 5, 16, 8, 4, 2, 1	8	16

Develop a Python program to take as input two positive integers n_1 and n_2 that defines a range of initial values for Collatz conjectures that start with n where $n_1 < n < n_2$ and output the maximum **Total Stopping Time** and the maximum **Maximum Value** among all those Collatz conjunctures.

Format:

Input: $n_1 \ n_2$ (space separated range for initial values n , where $n_1 < n < n_2$)

Output: Maximum of Total Stopping Times
 Maximum of Maximum Values

Sample:

Case 1

Input: 0 10

Output: 19
 52

Case 2

Input: 700 1000

Output: 178
 250504

Index No.	1701026
Name	Hala/Deen M.S
Group	E3
Date	15/02/2018

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

Develop a Python program to take four points on a 3D cartesian system as input and output if the points are colinear (on the same line), coplanar (on the same plane) or not coplanar. An example image of four coplanar points is given below.



Hints:

Let P, Q, R and S be the four points, if the points are coplanar then $\vec{PQ} \cdot (\vec{PR} \times \vec{PS}) = 0$ where "." represents the dot product and "x" represents the cross product.

Let $\vec{p} = a\vec{i} + b\vec{j} + c\vec{k}$ and $\vec{q} = d\vec{i} + e\vec{j} + f\vec{k}$; then the cross product is: $\vec{p} \times \vec{q} = (bf - ce)\vec{i} + (cd - af)\vec{j} + (ae - bd)\vec{k}$, and the dot product is: $\vec{p} \cdot \vec{q} = ad + be + cf$. Here, $\vec{i}, \vec{j}$ and $\vec{k}$ represent the unit vectors along the x, y and z axes, respectively, as used in standard notation.

Format:

Input: Four points P, Q, R and S on four lines, each point by its x, y, z coordinates, separated by space.

Output: "On the same plane" if the points are on the same plane and "Not on the same plane" if the points are not on the same plane. If the points are colinear, output "All the points are colinear"

Sample:

Case 1

Input: 1 2 3
4 5 6
7 8 9
11 12 13

Output: All the points are colinear

Case 2

Input: 1 2 3
4 5 6
6 7 7
7 8 7

Output: On the same plane

Case 3

Input: 1 2 3
4 5 6
7 8 7
6 6 1

Output: Not on the same plane

Index No.	
Name	
Group	
Date	

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

A human DNA can be represented by a sequence of letters containing A, C, G and T. For example, ACCGTTAGTAG.

Out of these, A and T are complementary to each other; similarly, G and C are complementary to each other (this means, A can form strong Hydrogen bonds with T and G can form strong Hydrogen bonds with C).

When a child is born, usually mutations (alterations) occur in the child DNA. These mutations can occur in the following manner:

- A letter in the parent DNA sequence can get replaced by another letter in the child DNA sequence
- A letter in the parent DNA sequence can get replaced by a gap in the child DNA sequence

We can score how different a child DNA is from the parent DNA using the following scheme.

1. When a letter in the parent DNA sequence get replaced by its complementary letter, we allocate a score of 2.
2. When a letter in the parent DNA sequence get replaced by another letter which is not complementary, we allocate a score of 4.
3. When a letter in the parent DNA sequence get replaced by a gap, we allocate a score of 6.

By adding the above scores we can compute the final **mutation score** that indicates how different the child DNA is from its parent DNA. Develop a Python program which will take as input a parent DNA sequence and a child DNA sequence and output the mutation score of the child DNA.

Format:

Input: line 1: Parent DNA sequence
line 2: Child DNA sequence (Here a gap is indicated by an underscore "_")

Output: The mutation score

Sample:

Case 1

Input: ACCGTGAATT
AGG_AGAATT
Output: 12

Case 2

Input: TGTAACCTTATTG
TGTT__TT_ATG
Output: 22

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

CS1032 Programming Fundamentals

2017 Intake, Jan - April 2018

Programming Problem 1

Time: 1 hour only

P6 [E9, Fri 8.15]

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

The n -th Fibonacci number can be defined as $F(n) = F(n-1) + F(n-2)$, $n > 1$, where $F(0)=0$ and $F(1)=1$.

A Fibonacci number is said to be *square-free* if it is not divisible by the square of a prime number. For example, the 30th Fibonacci number is 832040 but it is not square-free.

Develop a Python program that takes a positive integer n as input and outputs whether the n th Fibonacci number is square-free or not.

Format:

Input: A positive integer n

Output: YES if the n th Fibonacci number is square-free, NO otherwise

Sample:

Case 1

Input: 6

Output: NO

Case 2

Input: 15

Output: YES

Index No.	1701001
Name	V. G. S. Dargadho
Group	E-2
Date	16.02.2017

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

The Erdős–Gallai theorem says that, a sequence of positive integers $d_1, d_2, \dots, d_n$ that satisfies the following two conditions is *graphic*.

- Condition 1: The summation $\sum_{i=1}^n d_i$ is even.

- Condition 2: For every k where $1 \leq k \leq n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

where $\min(x, y)$ returns the minimum between x and y .

Develop a Python program that takes a sequence of positive integers as input and outputs whether it is graphic or not.

Format:

Input: Comma separated sequence of positive integers

Output: "Graphic" if the input sequence is graphic, "Non-Graphic" otherwise

Sample:

Case 1

Input: 2,2,2,1,1

Output: Graphic

Case 2

Input: 2,1,2,1,1

Output: Non-Graphic

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

A sequence of numbers is defined by $(n+1)(n+2)x_n = 3(n+1)(n+3)x_{n-1} - 2(n+2)(n+3)x_{n-2}$ where x_n is called the n^{th} number in the sequence and $x_1 = 4$ and $x_0 = 0$.

Develop a Python program to take as input a list of positive integers (each representing one possible value for n) and output the n^{th} number in the above sequence for each n in the list. The input list ends with a negative integer, which is used only to detect the end of input.

Format:

Input: A list of positive integers each on a new line, ending with a negative integer.

Output: The n^{th} number in the sequence for each positive integer n in the input

Sample:**Case 1**

Input:

10

12

25

17

-1

Output:

13299

61425

939524068

2621420

Case 2

Input:

15

21

13

-5

Output:

589806

50331624

131056

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

In mathematics, the *look-and-say sequence* is a sequence of positive integers where the first term is pre-defined and the n^{th} term ($n > 1$) in the sequence is determined by the $(n-1)^{\text{th}}$ term. If the first term is 1, the sequence is:

1, 11, 21, 1211, 111221, 312211, 13112221, 1113213211, ...

To generate the n^{th} term, just "look and say or read" the digits of the previous $(n-1)^{\text{th}}$ term.

Example: when 1 is the first term:

- 1 is read off as "one 1", so the next term is 11
- 11 is read off as "two 1s", so next term is 21
- 21 is read off as "one 2, then one 1", so next term is 1211
- 1211 is read off as "one 1, then one 2, then two 1s", so the next term is 111221
- The sequence continues as above

Develop a Python program that takes two integers m and n as input (in two lines) and output the n -th term of the look-and-say sequence whose first term is m .

Format:

Input: integer m
integer n

Output: n^{th} term in the look-and-say sequence where the first term is m

Sample:**Case 1**

Input: 5
5
Output: 132115

Case 2

Input: 6
4
Output: 3116

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Index No.	1702906
Name	S.N.S.D. Karunakaran
Group	EY
Date	2018, 02, 19

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

The *Fibonacci Sequence* includes the *Fibonacci Numbers* from $n = 0$ onwards for positive integers n . The n^{th} Fibonacci number $F(n)$ is defined as follows.

$$F(n) = F(n-1) + F(n-2), \text{ for } n > 1 \text{ where, } F(0) = 0 \text{ and } F(1) = 1$$

The Fibonacci sequence, therefore, can be listed as: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,

The *Pisano Period* is defined as follows. "For any integer n , the sequence of Fibonacci numbers F_i taken modulo n is periodic. The Pisano period, denoted by $\pi(n)$, is the least length of the period of this sequence".

Example: Pisano period for $n=3$

Fibonacci Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,

Fibonacci Sequence Modulo 3: 0, 1, 1, 2, 0, 2, 2, 1, 0, 1, 1, 2,

For $n=3$, the pattern 0,1,1,2,0,2,2,1 repeats
pattern length=8 $\rightarrow$ Pisano Period ($\pi(3)$) = 8

Develop a Python program that takes as input an integer n , ($1 < n < 10$) and outputs the Pisano period for n . (You may have to find the Fibonacci sequence including at least 50 Fibonacci numbers before finding the Pisano period).

Format:

Input: integer n , $1 < n < 10$

Output: The Pisano period for n

Sample:Case 1

Input: 8

Output: 12

Case 2

Input: 6

Output: 24

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

$$n[0:1]$$

$$n[0] + m +$$

$$n[0:0] + m + n[0:]$$

$$n[0:1] + m + n[1:]$$

Problem

Let n and k be +ve integers. Let m be another +ve integer such that $0 \leq m \leq 9$. We say a n is ' k -super' w.r.t. m if all the numbers we get from placing m either in the middle of two digits of n or in the beginning or the end of n is divisible by 2^k .

Ex

Assume you are given $n = 1408$ and $m = 8$.

Then we can get following numbers by replacing m in n
 $81408, 18408, 14808, 14088, 14008$

Since all the above numbers are divisible by $8 = 2^3$ we call 1408 a 3-super number w.r.t. 8.

Q

Develop a python program to take numbers n, k and m as input and output TRUE if n is k -super w.r.t. m and FALSE otherwise.

Section C

1. If the vectors $\underline{a}$ and $\underline{b}$ are not parallel, simplify the following

(i) $(2\underline{a} + \underline{b}) \wedge (\underline{a} - 2\underline{b})$

(ii) What is the direction of the vector in (a).

2. Relative to a fixed origin O the point A has position vector $\langle -2, 4, 7 \rangle$ and $\langle -1, 3, 8 \rangle$. Given that the angle PBA is θ show that $\cos \theta = \frac{1}{3}$.

3. Points $A \langle 2, 2, 5 \rangle, B \langle 1, -1, -4 \rangle, C \langle 3, 3, 10 \rangle$ and $D \langle 8, 6, 3 \rangle$ are the base.

(i) Calculate the lengths AB and AC and the angle BAC .

(ii) Show that $\overline{AD}$ is perpendicular to both $\overline{AB}$ and $\overline{AC}$.

(iii) Calculate the volume of the pyramid $ABCD$.
 [Volume of the pyramid is $V = \frac{1}{3} \times \text{base area} \times \text{perpendicular height}$]

Format:

Input: k, m and n separated by spaces. All the values are +ve integers. $4 \leq |n| \leq 100000$ where $|n|$ is the number of digits in n .

Output: **TRUE** if n is k -super w.r.t. m and **FALSE** if not.

Sample:

Case 1

Input: 3 8 1408

Output: TRUE

Case 2

~~Input: 3 0 4340~~

Input: 3 0 43406193657965875880

Output: TRUE

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor. **Example:** If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py".
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

The **Kronecker product** is a special type of matrix multiplication in mathematics. It is an operation on two matrices, resulting in a block matrix.

If A is an $m \times n$ matrix and B is a $p \times q$ matrix, then the Kronecker product, denoted by $A \otimes B$, is the block matrix of size $mp \times nq$.

$$A \otimes B = \begin{bmatrix} a_{11}B & \dots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \dots & a_{mn}B \end{bmatrix}$$

Here is an example Kronecker product computation between two 2×2 square matrices.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \otimes \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix} & 2 \cdot \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix} \\ 3 \cdot \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix} & 4 \cdot \begin{bmatrix} 0 & 5 \\ 6 & 7 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \cdot 0 & 1 \cdot 5 & 2 \cdot 0 & 2 \cdot 5 \\ 1 \cdot 6 & 1 \cdot 7 & 2 \cdot 6 & 2 \cdot 7 \\ 3 \cdot 0 & 3 \cdot 5 & 4 \cdot 0 & 4 \cdot 5 \\ 3 \cdot 6 & 3 \cdot 7 & 4 \cdot 6 & 4 \cdot 7 \end{bmatrix} = \begin{bmatrix} 0 & 5 & 0 & 10 \\ 6 & 7 & 12 & 14 \\ 0 & 15 & 0 & 20 \\ 18 & 21 & 24 & 28 \end{bmatrix}$$

Develop a Python program to take as input two $n \times n$ square matrices A and B and output the Kronecker product $A \otimes B$. You are not allowed use libraries for matrix multiplication.

Format:

Input: n (number of rows/columns of the two input square matrices)
Input matrix A (comma separated list of elements, row-by-row)
Input matrix B (comma separated list of elements, row-by-row)

Output: Kronecker Product between A and B (row elements comma separated, each row in a line)

Sample:

Case 1

Input: 2
1, 2, 3, 4
0, 5, 6, 7
Output: 0, 5, 0, 10
6, 7, 12, 14
0, 15, 0, 20
18, 21, 24, 28

Case 2

Input: 2
1, 2, 5, 6
0, 0, 1, -1
Output: 0, 0, 0, 0
1, -1, 2, -2
0, 0, 0, 0
5, -5, 6, -6

Index No.	
Name	
Group	
Date	

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
3. The samples are given as test cases. *Your code will be evaluated on a different set of test cases.*

The Problem

Given a sequence of numbers, we can calculate the mean, median, mode and range as follows.

- Mean = (sum of the terms) ÷ (number of terms)
- Median = The middle term of the sequence when they are arranged in the ascending (increasing) order; if the sequence has an even number of terms, then median is the average of the middle two terms
- Mode = The term or terms that appear the most number of times in the sequence
- Range = maximum term – minimum term

Develop a Python program which will take a sequence of positive integers as input and outputs the mean, median, mode and the range of the input sequence. Note that *the input sequence may not be sorted* in any specific order.

Format:

Input: A space separated sequence of positive integers ending with -1 (-1 is not part of the sequence).

Output: Mean, median, mode and range of the input sequence. If the mean value is not an integer, then it should be rounded to two decimal places.

Sample:

Case 1

Input: 13 18 13 14 13 16 14 21 13 14 14 -1
Output: mean: 14.82
median: 14
mode: 13, 14
range: 8

Case 2

Input: 37 21 1 19 50 50 -1
Output: mean: 29.67
median: 29
mode: 50
range: 49

CS1032 Programming Fundamentals

2017 Intake, Jan - April 2018

Programming Problem 1

Time: 1 hour only

P14 [E6, Mon 3.15]

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

The Newman-Conway sequence is a sequence of numbers, where the n^{th} term is defined by:

$$P_n = P_{P_{n-1}} + P_{n-P_{n-1}} \text{ and } P_1 = P_2 = 1.$$

The first few terms of the sequence are 1, 1, 2, 2, 3, 4, 4, 4, 5, 6, ...

Develop a Python program to take as input a positive integer n , and output the n^{th} number in the Newman-Conway sequence.

Format:

Input: A positive integer n

Output: n^{th} term in the Newman-Conway sequence

Sample:

Case 1

Input: 10

Output: 6

Case 2

Input: 15264

Output: 7890

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

P-15

A Diophantine equation is a polynomial equation, usually in two or more unknowns, such that only the integer solutions are sought. For example, $a^2 + b^2 = c^2$ is such kind of equation where a, b and c are integers. Integer solutions are called Pythagorean triples for this case.

For the equation $a^2 + b^2 = c^2$, one integer solution is the Pythagorean triple $(3, 4, 5)$ which means $a=3$ $b=4$ $c=5$. This $(3, 4, 5)$ is called a primitive Pythagorean triple because it has no common divisor larger than 1.

The Pythagorean triple $(6, 8, 10)$ is another solution for $a^2 + b^2 = c^2$, but it is not a primitive Pythagorean triple because it has a common divisor of 2.

Develop a python program to take as input a positive integer n and output the number of primitive Pythagorean triples less than n .

A primitive Pythagorean triple (a, b, c) is said to be less than n if $a < n$, $b < n$ and $c < n$. Note that (a, b, c) and (b, a, c) should be considered equivalent, not as different triples.

Format :

Input: n (positive integer)

Output: the number of primitive Pythagorean triples (a, b, c) such that $a < n$, $b < n$ and $c < n$

Sample :

Case 1

Input: 20

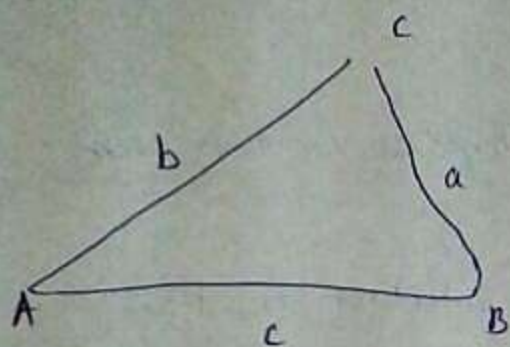
Output: 3

Case 2

Input: 100

Output: 16

Develop a python program that takes the (x,y) co-ordinates of a set of points how many of them can be used to form obtuse triangles.
An obtuse triangle is a triangle with one obtuse & (greater than 90°)



Hints:

Think of the cosine formula as well as the pythagorean theorem. The cosine formula states that $c^2 = a^2 + b^2 - 2ab \cos C$.

Format:

Input: A list of points on the xy plane, with x & y coordinates of each point separated by space on a new line. The last line of the input contains -1 (which indicates end of input).

Output: The number of obtuse triangles that can be found with the list of input points

Case 1

Input

0 0

0 1

1 0

-1 2

3 4

-1

Case 2

2 3

0 2

1 1

3 5

-1

Output: 5

2

CS1032 Programming Fundamentals

2017 Batch, Feb - March 2018

Programming Assignment 1

Time: 1 hour only

P17 [E10 (TT), Tue 1.15]

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor. *Example:* If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

Fibonacci numbers are numbers in the sequence 1, 1, 2, 3, 5, 8, 13 ... which is defined by $F_n = F_{n-1} + F_{n-2}$ where $F_1 = F_2 = 1$. Similarly we can define k-bonnaci numbers where for a given k , the number sequence is defined by $F_n = F_{n-1} + F_{n-2} + \dots + F_{n-k} = \sum_{i=1}^k F_{n-i}$ where $F_1 = F_2 = \dots = F_k = 1$.

You are given two integers, k and n as input. Develop a Python program to output the n^{th} k-bonnaci number.

Format:

Input: Two positive integers k and n separated by a space.

$$2 \leq k \leq 100$$

$$1 \leq n \leq 10000$$

Output: n^{th} k-bonnaci number

Sample:

Case 1

Input: 2 5
Output: 5

Case 2

Input: 13 39
Output: 402358273

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your *home* directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

A binary matrix is a matrix in which all entries are either 1 or 0. A square matrix is a matrix with the same number of rows and columns. A **permutation matrix** is a square binary matrix, which has only one entry of "1" in each row and column (and "0" in all other entries).

Example: The following is a 4×4 permutation matrix.

0	0	1	0
0	0	0	1
0	1	0	0
1	0	0	0

The above 4×4 matrix can be represented by a list of 16 comma separated values, row-wise, as:
0,0,1,0,0,0,0,1,0,1,0,0,1,0,0,0

Develop a Python program to take as input a positive integer n (where $n < 100$) and a list of $n \times n$ comma separated values and outputs whether the list represents a permutation matrix or not.

Format:

Input: Line 1: A positive integer n (where $n < 100$)
Line 2: A list of $n \times n$ comma separated values

Output: YES if the input list represents a permutation matrix and NO otherwise.

Sample:**Case 1**

Input: 3
1,0,0,1,0,1,1,1,-1

Output: NO

Case 2

Input: 5
1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,0,1,0,0,0,0,1

Output: YES

Index No.	170524T
Name	P. S. A. S. J. J.
Group	E 8
Date	21/02/2018

Instructions:

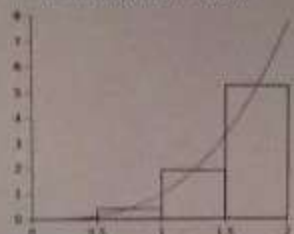
1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

Riemann integration is a numerical method of integrating a function f over a given finite period $[a, b]$. The function is divided into small sections where each section is approximately equal to a rectangle (when partition width is small). The total area is approximated by summation of each area segment.

Riemann summations are usually done with partitions of equal size. The interval $[a, b]$ is therefore divided into n subintervals, each of length $\Delta x = (b - a)/n$.

The points in the partition will then be, $a, a + \Delta x, a + 2\Delta x, \dots, a + (n-2)\Delta x, a + (n-1)\Delta x, b$.

Midpoint Riemann Sum

Among the four methods of Riemann, the *Midpoint Riemann Sum* approximates f at the midpoint of intervals, such as $f(a + \Delta x/2)$ for the first interval, $f(a + 3\Delta x/2)$ for the next one, and so on until $f(b - \Delta x/2)$.

The figure on the left shows the computation of the Midpoint Riemann sum of x^3 over $[0, 2]$ using 4 subintervals, each of length $\Delta x = (2 - 0)/4 = 0.5$.

Consider a polynomial function $f(x) = x^2 - 4x + 7$. The value of the function when $x = 1$ can be calculated as $f(1) = 1^2 - 4 \times 1 + 7 = 4$.

Develop a Python program to compute the Riemann integration, using Midpoint Riemann sum method, of a polynomial function f in a given range $[a, b]$. The polynomial function will be in the form of $f(x) = px^n + qx^{n-1} + rx^{n-2} + \dots$ (the coefficients $p, q, r, \dots$ are given as input, in the ascending order). The other inputs are the interval $[a, b]$ and the number of subintervals. The output should be with 2 decimal places.

Sample:**Case 1**

Input: 1, 2, 3

3, 4

100

Output: 45.00

- Coefficients p, q, r of polynomial function f
- interval $[a, b]$ where $a=1$ and $b=4$
- n , the number of subintervals

Case 2

Input: 1, 0, 2, 0, 5

10, 20

200

Output: 1104662.08

- Coefficients p, q, r, s, t of polynomial function f
- interval $[a, b]$ where $a=10$ and $b=20$
- n , the number of subintervals

[Fill the top-right box on this page and return this sheet at the end to the staff. Failure to do so is considered as non submission of the assignment. Do not write anything else unless asked by staff.]

Index No.	170586 N
Name	M.P.K. Silva
Group	P8
Date	21/02/2018

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

An integer number is said to be a "queer number" if the following two conditions are true.

1. When we take any two consecutive digits from the number, they form a prime.
2. The numbers that can be formed with two consecutive digits are all different from each other.

Examples:

- 43797 is a queer number because it satisfies both conditions (43, 37, 79, and 97 are all distinct primes)
- 11131 is not a queer number, because we can find 11 twice in 11131 which violates the condition (2).

Develop a Python program that takes two integers n_1 and n_2 , where $n_1 < n_2$, as input and outputs the number of queer numbers between n_1 and n_2 , inclusive of n_1 and n_2 .

Format:

Input: n_1 n_2 (positive integers n_1 and n_2 , separated by space; $n_1 < n_2$)

Output: the number of queer numbers between n_1 and n_2 , inclusive of n_1 and n_2 .

Sample:

Case 1

Input: 1452 7895

Output: 96

Case 2

Input: 50 1735

Output: 77

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

In a bakery shop, buns are served in packs of three sizes s_1 , s_2 and s_3 . Their prices are p_1 , p_2 and p_3 , respectively. They accept a customer order only if the order can be served in full packs. The shop owner always serves a customer order such that the total cost is minimized for the customer.

Example: Suppose the pack sizes are $s_1 = 6$, $s_2 = 9$ and $s_3 = 20$ and their prices are $p_1 = 10$, $p_2 = 16$ and $p_3 = 29$, respectively. If the customer ordered 67 buns, then it will be served at minimum cost as follows:

- 3 packs of size s_1 : $3 \times 6 = 18$ buns at cost $= 3 \times 10 = 30$
- 1 pack of size s_2 : $1 \times 9 = 9$ buns at cost $= 1 \times 16 = 16$
- 2 packs of size s_3 : $2 \times 20 = 40$ buns at cost $= 2 \times 29 = 58$
- Total of above: 67 buns at a cost of 104, which is the minimum cost
- Note: there can be other ways to fulfill the order of 67 buns, but at higher cost. For e.g., 3 packs of size s_2 ($3 \times 9 = 27$ buns) and 2 packs of size s_3 ($2 \times 20 = 40$ buns) will serve the order but at a higher cost of $3 \times 16 + 2 \times 29 = 48 + 58 = 106$.

Develop a Python program that takes as input the three pack sizes, their prices and the amount ordered by the customer and outputs the minimum cost for the order and the number of packs of each size in the order. If the order cannot be served, then output "Cannot serve order".

Format:

Input: Line 1: s_1 p_1 s_2 p_2 s_3 p_3 (size and price of 3 packs, separated by space)
Line 2: Amount ordered by customer

Output: Minimum price of the served order (or, "Cannot serve order")
[number of packs of size s_1 , s_2 and s_3 in the served order, separated by commas]

Sample:

Case 1

Input: 6 10 9 16 20 29
67
Output: 104
[3, 1, 2]

Case 2

Input: 6 10 9 14 20 28
19
Output: Cannot serve order

Index No.	170539X
Name	V. Sajeevan
Group	E7
Date	21/03/2018

Instructions:

1. Name your source file as "p1-<IndexNum>.py". You can work in VPL editor or IDLE or any other text editor.
 Example: If your Index No. is "170001C" then the name of your source file will be "p1-170001C.py"
2. If you use IDLE, save file in a folder named "p1" in your home directory. The file must be uploaded to VPL.
3. The samples are given as test cases. Your code will be evaluated on a different set of test cases.

The Problem

A Hadamard matrix is a square matrix whose elements are either 1 or (-1) and has the following property where I_n is the $n \times n$ identity matrix and H^T is the transpose of H .

$$HH^T = nI_n$$

Example:

The following matrix H is a Hadamard matrix as it has the above property $HH^T = nI_n$.

$$H = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\text{Transpose, } H^T = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$\text{Multiplication, } HH^T = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = 2 * \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$(AB)_{ij} = \sum_{k=1}^m A_{ik} B_{kj}$$

Formula of Matrix Multiplication

Develop a Python program to take a $n \times n$ square matrix as input and output whether the input matrix is Hadamard or not. You are not allowed use libraries for matrix multiplication.

Format:

Input: n (specifies n in the $n \times n$ square matrix)
 the matrix (each row in a line, elements in a row separated by commas)
 Output: "Hadamard" if the input matrix is a Hadamard matrix, "Not Hadamard" otherwise

Sample:

Case 1

Input: 2
 1, 1
 1, -1

Output: Hadamard

Case 2

Input: 2
 2, -1
 -1, -1

Output: Not Hadamard